

Learning 3D Gaussian Splatting from Degraded Images

Lee, Jungho

Department of Electrical and Electronic Engineering
Graduate School
Yonsei University

Learning 3D Gaussian Splatting from Degraded Images

Advisor Lee, Sangyoun

A Dissertation

Submitted to the Department of Electrical and Electronic Engineering
and the Committee on Graduate School of Yonsei University
in Partial Fulfillment of the
Requirements for the Degree of
Doctor of Philosophy in Electrical and Electronic Engineering

Lee, Jungho

August 2026

Learning 3D Gaussian Splatting from Degraded Images

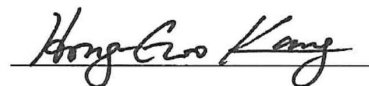
This Certifies that the Dissertation of Lee, Jungho is Approved.



Committee Chair Lee, Sangyoun



Committee Member Lee, Chungyong



Committee Member Kang, Hong-Goo



Committee Member Lee, Dogyoon



Committee Member Cho, Suhwan

Department of Electrical and Electronic Engineering

Graduate School

Yonsei University

August 2026

Contents

List of Figures	iv
List of Tables	viii
Abstract	x
1 Introduction	1
2 Related Work	7
2.1 Neural Rendering	7
2.2 Neural Rendering from Degraded Images	8
3 Preliminaries	9
3.1 3D Gaussian Splatting	9
3.2 3D Gaussian Splatting from Degraded Images	11
4 Leveraging Circle of Confusion for Gaussian Splatting from Defocused Images	14
4.1 Introduction	14
4.2 Preliminaries	18

4.2.1	Circle of Confusion	18
4.3	Method	20
4.3.1	CoCoGaussian Framework	20
4.3.2	Circle of Confusion from 3D Gaussians	23
4.3.3	Adaptive CoC Gaussian Generation	24
4.3.4	Optimization	26
4.4	Experiments	27
4.4.1	Rendering Results	28
4.4.2	Ablation Study	32
4.4.3	Experiments on All-in-Focus Images.	38
4.4.4	Customizable Depth of Field and Focus Plane	39
4.4.5	Computational Efficiency and Speed	41
4.4.6	CoC Visualization	42
4.4.7	Per-Scene Quantitative Results	44

5 Continuous Motion-Aware Gaussian Splatting from Motion-Blurred

	Images	48
5.1	Introduction	48
5.2	Preliminaries	52
5.2.1	Neural Ordinary Differential Equations	52
5.3	Method	54
5.3.1	CoMoGaussian Framework	54
5.3.2	Continuous Rigid Body Motion	57

5.3.3	Continuous Motion Refinement	59
5.3.4	Optimization	61
5.4	Experiments	62
5.4.1	Novel View Synthesis Results	64
5.4.2	Ablation Study	67
5.4.3	Camera Trajectory Visualization	70
5.4.4	Number of Poses on Camera Motion.	76
5.4.5	Orthogonal Regularization.	78
5.4.6	Training and Rendering Speed	79
5.4.7	Per-Scene Quantitative Results	80
5.4.8	Additional Qualitative Results	84
6	Conclusion	86
	Abstract (in Korean)	95

List of Figures

4.1	We propose CoCoGaussian, a novel framework for 3D scene reconstruction from defocused images. (a) With a large aperture, radiance from the focus plane appears as a small circle of confusion (green) on the image sensor, while radiance from greater depths results in larger circles (pink). (b) We visualize reconstructed Gaussians of CoCoGaussian, where the black dots indicate Gaussian means, while the gray dots represent Gaussian means forming the circle of confusion, which decreases in size near the focus plane. (c) The defocused image focuses on shallow depths, with circle size increasing with depth, as shown in (b). CoCoGaussian allows customization of defocused images by adjusting the depth of field or focus plane, while sharp images can be obtained by rendering without the circle of confusion.	15
4.2	(a) CoC sizes based on the position of object relative to the focus plane, and (b) CoC sizes based on the aperture size.	19

4.3	We take the camera position $\mathbf{x}_{cam}$ and the base Gaussian parameters μ_B , $\mathbf{s}_B$, and $\mathbf{q}_B$ as inputs to the MLP h_θ , which produces five outputs in total. (a) We set the depth $d(\mu_B)$ as the Euclidean distance between $\mathbf{x}_{cam}$ and μ_B . Using the output K from h_θ , the $d(\mu_B)$, and the learnable focus plane d_F , we apply Eq. (4.2) to determine the CoC diameter. The diameter, combined with the outputs β and $\mathbf{d}$ from h_θ , is then used in Eq. (4.4) to compute the offset values for μ_{CoC} . By adding these offsets to μ_B , we obtain the $\mathbf{G}_{CoC}$ means. (b) We obtain $\mathbf{s}_{CoC}$ and $\mathbf{q}_{CoC}$ by applying Eqs. (4.5) and (4.6) to $\delta\mathbf{s}_{CoC}$ and $\delta\mathbf{q}_{CoC}$, the outputs of h_θ . (c) Using μ_{CoC} , $\mathbf{s}_{CoC}$, and $\mathbf{q}_{CoC}$, we get the $\mathbf{G}_{CoC}$. Finally, we rasterize $\mathbf{G}_{CoC}$ along with $\mathbf{G}_B$ to produce $(M+1)$ images, and apply a weighted sum to generate the final defocused image.	22
4.4	Qualitative comparison on the Deblur-NeRF and DoF-NeRF datasets.	31
4.5	Qualitative Ablation of CoC Scaling Factor. The figures are from CUP and TOOLS scenes of Deblur-NeRF [1] real-world dataset.	35
4.6	Visualization of Aperture parameter and Focus Plane Customization. The top row of images decreases the aperture parameter K from left to right, while the bottom row moves the focus plane d_F further from the camera from left to right.	40
4.7	Gaussian Positions of Defocused Images. The black and gray dots indicate base and CoC Gaussians, respectively.	43
4.8	Gaussian Positions of All-in-Focus Images.	44

5.1	We propose CoMoGaussian, a novel framework for reconstructing 3D scenes from camera motion-blurred images. (a) Camera motion-blurred images are generated by the continuous movement of the camera during the exposure time. (b) The top image represents the camera poses obtained through CoMoGaussian, showing a continuous trajectory indicated by gradual change of colors. The bottom images are the rendering outputs from each camera pose. (c) By aggregating the rendered images obtained in step (b), we get the motion-blurred output image shown at the top, while the sharp output image is rendered from the given calibrated camera pose.	49
5.2	Pipeline of CoMoGaussian. (a) The given image index is embedded and passed through the encoders $\mathcal{E}_r$ and $\mathcal{E}_c$ to obtain latent features for the rigid body transformation and the CMR transformation. (b) CoMoGaussian solves the ODEs using the latent features and the neural derivatives f_ϕ and g_ψ , and obtains N transformed camera poses via Eq. (5.2). (c) Then, N images are rendered through Mip-Splatting [2], and the output motion-blurred image is obtained through the pixel-wise weighted-sum and scalar pixel mask (Sec. 5.3.4).	56
5.3	Qualitative comparison on the Deblur-NeRF [1] synthetic and real-world scenes, and ExbluRF [3] real-world scenes.	66
5.4	Camera Trajectory Visualization. Red cones stand for camera poses, and the images below are the output blurry images.	71

5.5	Comparison between the Neural ODE-based estimator and other estimators. Only the Neural ODE-based estimator exhibits a continuous camera trajectory.	72
5.6	Visualization of camera motion trajectories for sharp images.	73
5.7	Camera motion trajectory predicted by CoMoGaussian for input images with significant blur.	74
5.8	Camera motion trajectory predicted by CoMoGaussian for input images with moderate blur.	75
5.9	Performance variation based on the number of poses N used to construct the camera motion trajectory.	77
5.10	Qualitative comparison of results with and without orthogonal regularization. Despite similar PSNR values, the results with regularization capture more fine details.	79
5.11	Additional Qualitative Comparison on the Synthetic and Real-World Scenes.	85

List of Tables

4.1	Comparisons on Deblur-NeRF synthetic and real-world scene dataset. We evaluate the performance on three metrics (PSNR, SSIM, LPIPS). “*” denotes the results obtained by reproducing the released code. The orange and yellow cells respectively indicate the highest and second- highest value.	29
4.2	Comparisons on DoF-NeRF real-world scene dataset.	30
4.3	Ablation Results of CoCoGaussian.	33
4.4	Quantitative Results for the Number of CoC Gaussian Sets M . The orange and yellow cells respectively indicate the highest and second- highest values.	37
4.5	Results on NeRF-LLFF [4,5] Dataset.	38
4.6	Computational Efficiency and Speed. * indicates that the rendering speed is identical to that of the corresponding model.	41
4.7	Per-Scene Quantitative Results on Deblur-NeRF [1] Synthetic Dataset.	45
4.8	Per-Scene Quantitative Results on Deblur-NeRF [1] Real-World Dataset.	46

4.9	Per-Scene Quantitative Results on DoF-NeRF [6] Real-World Dataset.	47
5.1	Comparisons on Deblur-NeRF synthetic and real-world scene dataset.	
	“*” denotes the results obtained by reproducing the released code.	
	The orange and yellow cells respectively indicate the highest and second-highest value.	65
5.2	Comparisons on ExbluRF real-world scene dataset.	65
5.3	Ablation study on our main components.	67
5.4	Ablation study on the type of camera motion estimator.	68
5.5	Experimental results on NeRF-LLFF [4,5] dataset.	69
5.6	Ablation study on ODE solvers for continuous camera trajectory modeling (Deblur-NeRF ball scene).	70
5.7	Experimental results based on the number of poses N	76
5.8	Comparison of training and rendering speeds across various 3DGS-based methods. * indicates that the speed is identical to that of the corresponding model.	80
5.9	Per-Scene Quantitative Performance on Deblur-NeRF Real-World Scenes.	81
5.10	Per-Scene Quantitative Performance on Deblur-NeRF Synthetic Scenes.	82
5.11	Per-Scene Quantitative Performance on the ExbluRF Real-World Scenes.	83

Abstract

Learning 3D Gaussian Splatting from Degraded Images

Jungho Lee

The Graduate School

Yonsei University

Department of Electrical and Electronic Engineering

3D Gaussian Splatting (3DGS) has recently emerged as an efficient representation for real-time neural rendering, enabling high-quality novel view synthesis with significantly faster training and rendering compared to implicit methods. However, most existing approaches assume the availability of sharp input images, which is often unrealistic in practical scenarios. Real-world image acquisition frequently suffers from various degradations, such as defocus blur caused by limited depth of field and motion blur induced by camera movement during exposure. These degradations significantly hinder accurate 3D scene reconstruction and limit the applicability of neural rendering methods, particularly in real-world capture settings such as handheld 3D scanning, robotic perception, drone-based inspection, digital twin construction, and immersive AR/VR content generation.

In this thesis, we investigate the problem of learning 3D Gaussian Splatting from degraded images and propose physically grounded frameworks that enable accurate 3D reconstruction from blurred observations. First, we introduce a defocus-aware

Gaussian splatting framework that models the Circle of Confusion (CoC) based on photographic principles, allowing the system to reconstruct precise 3D scenes using only defocused images while enabling controllable depth-of-field rendering. Second, we address motion blur caused by camera movement during exposure by modeling continuous camera trajectories using neural ordinary differential equations (ODEs). By incorporating continuous rigid-body motion and motion refinement transformations, the proposed framework accurately captures motion blur formation and reconstructs sharp 3D scenes from motion-blurred images. Together, these approaches demonstrate that explicit modeling of real-world image formation processes is essential for robust neural rendering under non-ideal conditions. Extensive experiments on multiple benchmark datasets show that the proposed methods achieve state-of-the-art performance in reconstructing sharp 3D scenes and synthesizing high-quality novel views from degraded images. This thesis highlights the importance of integrating physical imaging principles with efficient explicit scene representations, paving the way toward practical 3D reconstruction systems that operate reliably in real-world imaging conditions. In the future, the proposed degradation-aware 3DGS frameworks can be used as core components of industrial 3D reconstruction and novel-view synthesis pipelines for digital twins, robotic and drone-based inspection, immersive AR/VR systems, virtual production, and autonomous perception, where robust scene modeling from imperfect visual data is essential for scalable deployment.

Keywords : 3D Neural Rendering; Gaussian Splatting; Image Degradation.

Chapter 1

Introduction

Recent advances in neural rendering have significantly improved the quality of 3D scene reconstruction and novel view synthesis from multi-view images. In particular, the introduction of Neural Radiance Fields (NeRF) [4] has enabled photo-realistic rendering of complex scenes by learning implicit volumetric representations from images captured at different viewpoints. However, NeRF relies on computationally expensive ray-marching processes, which limits its applicability to real-time applications.

To address this limitation, 3D Gaussian Splatting (3DGS) [7] has recently emerged as an efficient alternative for neural scene representation. Instead of representing scenes implicitly with neural networks, 3DGS explicitly models a scene using a collection of 3D Gaussians defined by their spatial positions, covariances, opacities, and appearance coefficients. By leveraging differentiable rasterization, 3DGS enables significantly faster training and rendering while maintaining high-quality novel view

synthesis. As a result, it has become a promising foundation for real-time neural rendering systems.

The practical impact of 3DGS is particularly significant because many emerging industrial applications require fast and high-quality 3D reconstruction from image observations. For example, digital twin construction for factories, construction sites, and urban environments requires scalable reconstruction of real-world spaces for monitoring, simulation, and visualization. Robotic and drone-based inspection systems require accurate 3D scene representations to analyze infrastructure, facilities, and hard-to-access environments. Immersive AR/VR, virtual production, telepresence, and spatial computing applications require real-time novel view synthesis and realistic scene visualization from captured imagery. In these scenarios, 3DGS can serve as a core representation for converting visual data into interactive 3D content. However, unlike controlled laboratory settings, images in such practical pipelines are often captured using handheld cameras, mobile robots, drones, or moving platforms under uncontrolled lighting, focus, and motion conditions. Therefore, improving the robustness of 3DGS to imperfect image acquisition is essential for deploying neural rendering systems in real-world industrial environments.

Despite these advances, most neural rendering methods including both NeRF and 3DGS assume that input images are sharp and free from degradations. In practice, however, images captured in real-world environments often contain various forms of degradation. Two of the most common types are defocus blur, caused by the limited depth of field in optical systems, and motion blur, caused by camera movement during

the exposure time. These degradations distort the captured radiance information and make it difficult to accurately reconstruct the underlying 3D scene structure.

Such degradations are not exceptional artifacts but common obstacles in practical 3D capture scenarios. Defocus blur frequently occurs when cameras operate with a shallow depth of field, when objects exist at different depths, or when the capture device cannot maintain an accurate focus plane during acquisition. This is common in close-range object digitization, indoor scanning, industrial inspection, and AR/VR content capture, where scenes may contain both near and far structures. Motion blur frequently occurs when cameras are mounted on moving platforms, such as drones, vehicles, or robots, or when handheld devices are used in low-light environments requiring longer exposure times. In industrial deployment, requiring perfectly sharp images would often increase capture time, require expensive stabilization hardware, or force repeated data acquisition. Therefore, a reconstruction system that can learn reliable 3D representations directly from degraded observations can substantially reduce the practical constraints of real-world 3D capture.

Defocus blur occurs when scene points lie outside the focus plane of the camera, causing light rays to spread on the image sensor and form a circular blur pattern known as the Circle of Confusion (CoC). As the distance between the scene point and the focus plane increases, the size of the CoC grows, resulting in spatially varying blur across the image. Conventional neural rendering methods are not designed to explicitly model this optical phenomenon, which leads to inaccurate scene reconstruction when defocused images are used as input. Previous studies [8–10] have attempted to

address this problem using learning-based blur kernels or implicit representations, but they often fail to incorporate the physical principles governing defocus formation.

Motion blur presents another major challenge for neural rendering systems. During image acquisition, camera motion within the exposure interval integrates scene radiance along a continuous camera trajectory, producing blurred observations. Most existing approaches [8–10] model motion blur using simplified blur kernels or discrete camera pose interpolation. However, these approximations often fail to capture the continuous nature of camera motion, resulting in inaccurate blur modeling and degraded reconstruction performance.

In this thesis, we investigate the problem of learning 3D Gaussian Splatting from degraded images, with the goal of enabling accurate 3D scene reconstruction and novel view synthesis under realistic imaging conditions. Specifically, we focus on two major types of degradation—defocus blur and motion blur—and propose physically grounded frameworks that explicitly model the image formation processes responsible for these degradations.

First, we introduce a defocus-aware Gaussian Splatting framework that models the CoC directly within the 3D Gaussian representation. By estimating the CoC diameter based on scene depth and camera parameters, and generating additional Gaussian sets that represent the spatial distribution of the CoC, the proposed approach can reconstruct accurate 3D scenes using only defocused images. This framework also enables controllable depth-of-field rendering by learning aperture and focus-plane parameters during training.

Second, we propose a motion-aware Gaussian Splatting framework that models camera motion blur by estimating continuous camera trajectories during the exposure time. To capture the continuous nature of camera motion, we leverage neural ordinary differential equations (ODEs) [11] to represent the temporal dynamics of the camera trajectory. Furthermore, we introduce a continuous motion refinement transformation that enhances rigid-body motion modeling and improves the accuracy of blur approximation. By integrating these components within a 3DGS framework, our method reconstructs sharp 3D scenes from motion-blurred images while maintaining real-time rendering capability.

Through extensive experiments on multiple benchmark datasets, we demonstrate that the proposed methods achieve state-of-the-art performance for neural rendering from degraded images. The results show that explicitly modeling the physical image formation processes underlying real-world degradations significantly improves reconstruction accuracy and rendering quality.

The contributions of this thesis can be summarized as follows:

- We investigate the problem of reconstructing 3D scenes using 3D Gaussian Splatting from degraded images captured under realistic imaging conditions.
- We propose a Circle of Confusion-aware Gaussian Splatting framework that enables accurate 3D reconstruction from defocused images.
- We introduce a continuous motion-aware Gaussian Splatting framework that models motion blur using neural ODE-based camera trajectory estimation.

- We demonstrate through extensive experiments that physically grounded modeling of image degradations significantly improves neural rendering performance.

Overall, this thesis advances neural rendering toward more practical real-world applications by enabling robust 3D scene reconstruction from degraded images. By reducing the dependence on perfectly sharp input images, the proposed frameworks contribute to the development of deployable 3D reconstruction systems that can operate under the imperfect imaging conditions commonly encountered in real-world industrial capture pipelines.

Chapter 2

Related Work

2.1 Neural Rendering

Neural rendering has seen explode in the fields of computer graphics and vision area thanks to the emergence of ray tracing-based neural radiance fields (NeRF) [4], which provide realistic rendering quality from 3D scenes. NeRF has led to a wide range of studies, including TensoRF [12], Plenoxel [13], and Plenotree [14], which have aimed at overcoming slow rendering speed. Among these advancements, the recent emergence of 3D Gaussian splatting (3DGS) [7], which offers remarkable performance along with fast training and rendering speed, has further accelerated research in the neural rendering. Additionally, there has been active research focusing on the non-ideal conditions of given images, such as sparse-view images [15–17], and the absence of camera parameters [18,19]. Moreover, there has been significant research attention on non-ideal conditions inherent to the images themselves, such as

low-light [20,21], blur [1,9,10,22–25]. Recently, neural rendering from blurry images has attracted attention due to its practical applicability.

2.2 Neural Rendering from Degraded Images

Deblur-NeRF [1] firstly introduced the deblurred neural radiance fields by importing the blind-deblurring mechanism into the NeRF framework. They introduce specific blur kernel in front of the NeRF framework imitating the blind deblurring in 2D image deblurring area. After the emergence of [1], several attempts have been proposed to model the precise blur kernel with various types of neural rendering baseline, such as TensorRF [12], and 3DGS [7]. DP-NeRF [10] proposes rigid blur kernel that predict the camera motion during image acquisition process as 3D rigid body motion to preserve the geometric consistency across the scene. BAD-NeRF [23] and BAD-Gaussians [25] similarly predict blur kernel as camera motions based on NeRF [4] and 3DGS [7], which assume the simple camera motion and interpolate them between predicted initial and final poses and design simple spline-based methods. Deblurring 3DGS [8] adjusts Gaussian parameters like rotation and scaling to generate blurry images during training, and BAGS [9] proposes CNN-based multi-scale blur-agnostic degradation kernel with blur masking that indicates the blurred areas. In this paper, we implement the continuous motion trajectory that forms camera motion blur in 3D using a neural ODE, which fundamentally differentiates our approach from existing methods.

Chapter 3

Preliminaries

3.1 3D Gaussian Splatting

3D Gaussian Splatting (3DGS) [7] represents a scene as a set of anisotropic 3D Gaussian primitives. Each Gaussian models a small region of the scene and is defined by its mean position $\mu \in \mathbb{R}^3$ and covariance matrix $\Sigma \in \mathbb{R}^{3 \times 3}$. A Gaussian function centered at μ is defined as

$$G(x) = \exp \left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu) \right), \quad (3.1)$$

where x denotes a point in the 3D space. The covariance matrix Σ controls the spatial extent and orientation of the Gaussian, allowing it to represent anisotropic shapes in the scene.

For stable optimization, the covariance matrix is commonly parameterized using

a scaling matrix S and rotation matrix R as

$$\Sigma = RSS^\top R^\top, \quad (3.2)$$

where S controls the size of the Gaussian along three principal axes and R controls its orientation.

During rendering, each Gaussian defined in the world coordinate system must be projected onto the image plane. The projection transforms the 3D covariance into a 2D covariance that describes the elliptical footprint of the Gaussian on the image plane. This transformation can be approximated as

$$\Sigma_{2D} = JW\Sigma W^\top J^\top, \quad (3.3)$$

where W denotes the camera transformation matrix and J is the Jacobian of the projection function.

Each Gaussian also stores an opacity value α and view-dependent color parameters, typically represented using spherical harmonics coefficients. When rendering an image, the contributions of multiple Gaussians are composited along the viewing direction using alpha blending. The final pixel color C_p is computed as

$$C_p = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (3.4)$$

where c_i and α_i denote the color and opacity of the i -th Gaussian. This equation

accumulates the contributions of Gaussians in a front-to-back order while accounting for occlusions.

Compared with volumetric rendering approaches such as NeRF [4], 3DGS enables significantly faster rendering because it relies on rasterization instead of ray marching. This efficiency makes it suitable for real-time neural rendering applications.

3.2 3D Gaussian Splatting from Degraded Images

Most neural rendering methods assume that input images are sharp. However, images captured in real-world environments often contain degradations introduced during the image acquisition process. Two common types of degradation are *defocus blur* and *motion blur*.

Defocus blur occurs when scene points lie outside the focus plane of the camera. In optical systems with a finite aperture, light rays from out-of-focus points spread across the image sensor, forming a circular pattern known as the *Circle of Confusion* (*CoC*). The diameter of the CoC depends on the depth of the scene point and the camera parameters. A commonly used approximation of the CoC diameter is

$$\sigma \approx K \left| \frac{1}{d} - \frac{1}{d_F} \right|, \quad (3.5)$$

where d denotes the depth of the scene point, d_F represents the focus plane distance, and $K = fD$ is a constant determined by the focal length f and aperture diameter D . This relationship indicates that the blur size increases as the distance between

the scene point and the focus plane becomes larger.

Motion blur arises when the camera moves during the exposure time. Instead of capturing the scene at a single camera pose, the sensor integrates radiance over a continuous camera trajectory. As a result, the observed blurry image can be interpreted as the temporal integration of multiple latent sharp images:

$$I_{blur} = \int_0^T I_{\tau} d\tau, \quad (3.6)$$

where I_{τ} denotes the image captured at time τ during the exposure interval T . In practice, this continuous integration is approximated using discrete samples along the camera trajectory:

$$I_{blur} \approx \sum_{i=1}^N w_i I_{\tau_i}, \quad \sum_{i=1}^N w_i = 1, \quad (3.7)$$

where I_{τ_i} represents the image rendered from the camera pose at time τ_i , and w_i denotes the corresponding weight.

From the perspective of 3DGS, these degraded images can be interpreted as the result of aggregating multiple renderings generated under different imaging conditions. For example, defocus blur can be modeled by distributing Gaussian primitives according to the CoC, while motion blur can be modeled by rendering the scene along a sequence of camera poses during the exposure time.

Building on these observations, recent studies have incorporated physically motivated degradation models into Gaussian splatting frameworks to enable accurate 3D

scene reconstruction from degraded observations. In this thesis, we build upon these ideas and focus on learning 3DGS from degraded images by explicitly modeling both defocus blur and motion blur in the image formation process.

Chapter 4

Leveraging Circle of Confusion for Gaussian Splatting from Defocused Images

4.1 Introduction

The emergence of the Neural Radiance Field (NeRF) [4] has brought significant attention to 3D scene representation for photo-realistic novel view synthesis. They reconstruct 3D scenes from images captured from multiple views and render images of unseen views. However, since NeRF models 3D scenes as implicit neural representations through a ray tracing-based approach, they suffer from inefficient memory usage. In contrast, a rasterization-based method, 3D Gaussian Splatting (3DGS) [7] explicitly models 3D scenes using independent 3D Gaussians with different 3D means, covariances, opacities, and spherical harmonic coefficients. The tile-based rasterizer of 3DGS applies alpha blending to Gaussian splats sorted by visibility order, enabling fast training, rendering, and efficient memory usage.

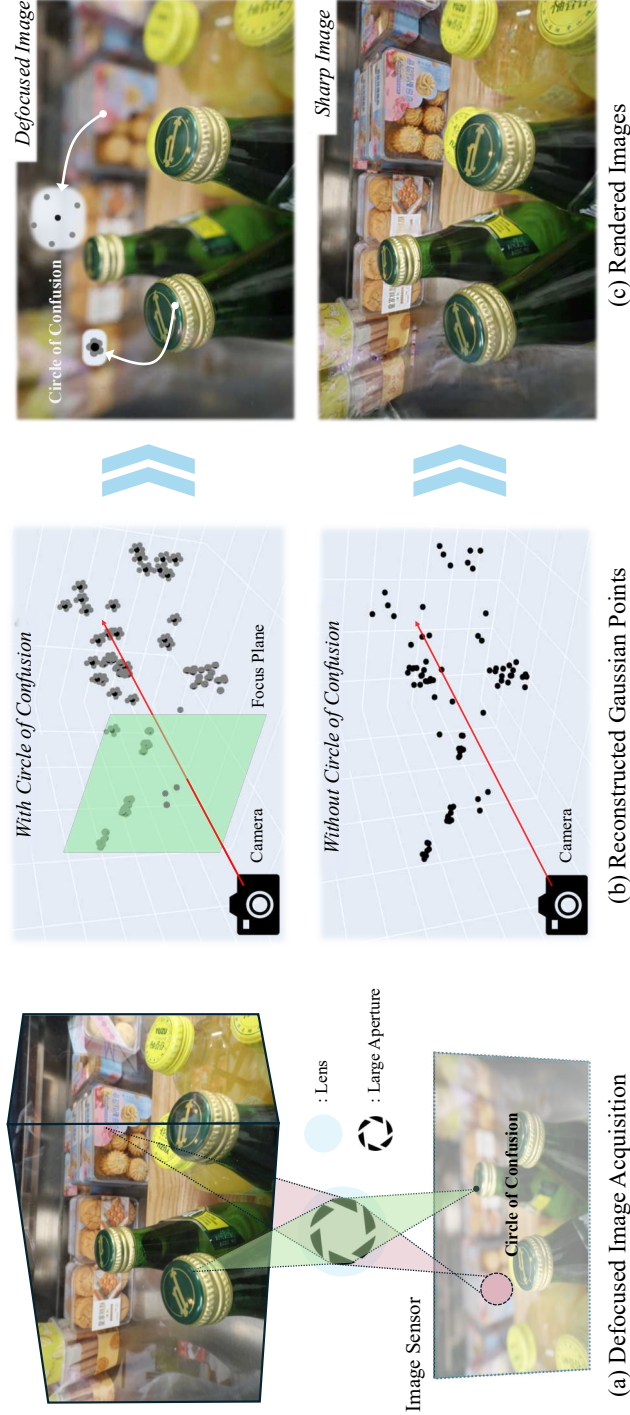


Figure 4.1: We propose CoCoGaussian, a novel framework for 3D scene reconstruction from defocused images. (a) With a large aperture, radiance from the focus plane appears as a small circle of confusion (**green**) on the image sensor, while radiance from greater depths results in larger circles (**pink**). (b) We visualize reconstructed Gaussians of CoCoGaussian, where the black dots indicate Gaussian means, while the gray dots represent Gaussian means forming the circle of confusion, which decreases in size near the focus plane. (c) The defocused image focuses on shallow depths, with circle size increasing with depth, as shown in (b). CoCoGaussian allows customization of defocused images by adjusting the depth of field or focus plane, while sharp images can be obtained by rendering without the circle of confusion.

Both NeRF [4] and 3DGS [7] rely on sharp images to represent 3D scenes accurately, which is a highly ideal assumption. In real-world settings, various factors (*e.g.*, finite depth of field and camera motion blur) can hinder the capture of sharp images, leading to image degradation. Achieving a perfectly sharp image requires every part of the scene to be in focus, namely all-in-focus, which calls for a large depth of field. However, obtaining a large depth of field necessitates using a small aperture, which in turn requires a longer exposure time to allow enough light to enter the camera. During this extended exposure, even slight movements of a handheld camera can introduce motion blur. To prevent motion blur, the aperture needs to be widened, but this comes at the cost of a shallower depth of field. As a result, areas outside the focus plane appear blurred due to defocus. Defocus blur is closely related to the concept of the Circle of Confusion (CoC); as shown in Fig. 4.1, when a subject is positioned away from the focus plane, the radiances from the subject pass through the aperture and are mapped onto the image sensor in a circular pattern, resulting in shallow depth of field and defocus blur [26]. However, despite these challenges, recent research in 3D scene representation has rarely focused on addressing defocus blur directly. In this paper, we propose a method to represent 3D scenes using only images with defocus blur, tackling the CoC-related challenges posed by real-world image acquisition scenarios.

Recently, there has been increasing interest in reconstructing 3D scenes using only degraded images, with a focus on addressing challenges caused by camera motion blur and defocus blur. Deblur-NeRF [1] is the first study to take on this task, adopting

a blind image deblurring approaches [27–29]. Follow-up studies [8–10,22–24,30] have expanded this approach beyond ray-tracing to rasterization-based methods. However, many of these methods depend heavily on learning-based strategies and fail to incorporate photographic principles that accurately capture how defocus occurs in real-world scenarios. While DoF-NeRF [6] adopts the concept of CoC to model defocus blur in 3D scene representations, it relies on an implicit representation, which leads to slow rendering speeds, making it less suitable for real-time applications.

In this paper, we propose CoCoGaussian, which leverages a physically grounded photographic defocus principles to represent 3D scenes. Our method incorporates an aperture to calculate the CoC diameter based on depth and learnable aperture information using 3D Gaussians, accurately modeling shallow depth of field images. Using the CoC diameter and the 3D Gaussian, we generate multiple Gaussians to form the CoC shape, as illustrated in Fig. 4.1. However, when objects with refraction or reflection exist in the scene, the depth obtained through the Gaussians may become unreliable. To address this, we propose a method to keep the set of generated Gaussians within the CoC radius, introducing a learnable scaling factor to increase flexibility and reduce dependence on estimated depth in the CoC modeling. As a result, our model combines physically grounded defocus principles with an adaptive CoC modeling that effectively handles uncertain depths, allowing it to reconstruct sharp 3D scenes using only defocused images. Additionally, our approach has an advantage of learning aperture and focus plane information, enabling dynamic control over depth of field and flexible adjustment of the focus plane. This feature allows

for highly customizable scene visualization, adapting to various focus and depth requirements.

To demonstrate the effectiveness of our model, we conduct extensive experiments on the benchmarks: the Deblur-NeRF [1] dataset and the DoF-NeRF [6] real-world dataset. Our contributions can be summarized as follows:

- CoCoGaussian models the CoC at the 3D Gaussian level, reconstructing the precise 3D scene and enabling sharp novel view synthesis from defocused images.
- We propose an adaptive learning approach that robustly models the CoC even with unreliable depth information.
- CoCoGaussian enables the customizable depth of field and flexible focus plane adjustment during rendering by learning aperture and focus plane information.
- CoCoGaussian achieves state-of-the-art performance, both quantitatively and qualitatively.

4.2 Preliminaries

4.2.1 Circle of Confusion

In this subsection, we explain the principles behind the generation of the CoC based on the focus plane and aperture size. As shown in Fig. 4.2 (a), when a subject is precisely located on the focus plane, the radiance emitted from a point on the

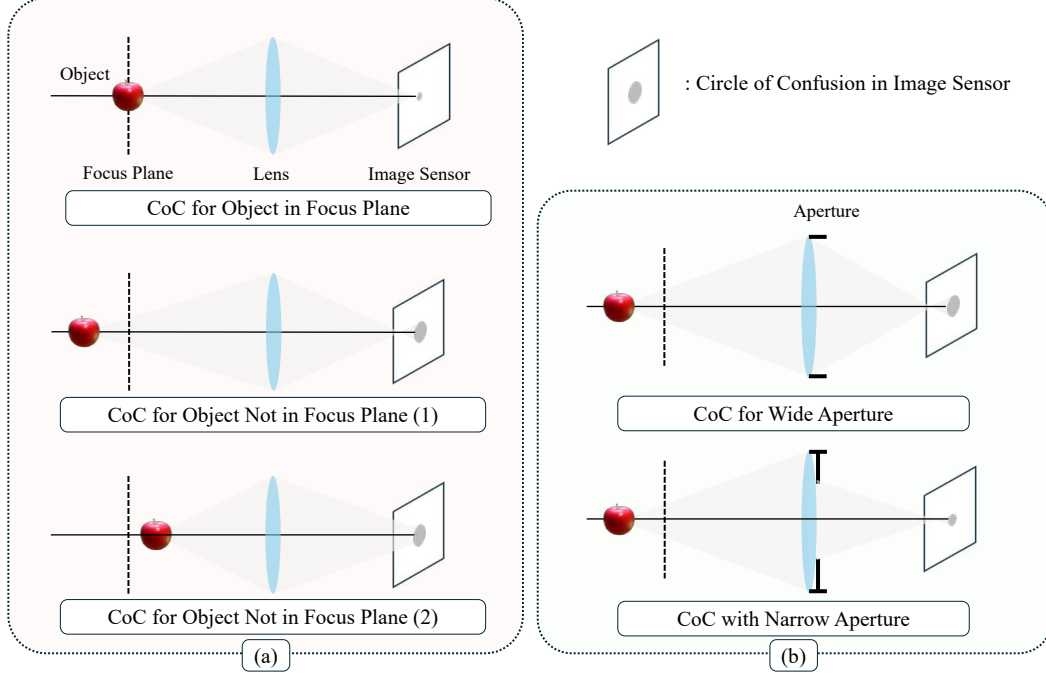


Figure 4.2: **(a)** CoC sizes based on the position of object relative to the focus plane, and **(b)** CoC sizes based on the aperture size.

subject is projected onto the image sensor as a single point. However, when the subject is positioned away from the focus plane, the radiance passes through the lens and forms a circular point spread function on the image sensor. As this circle increases in size, the defocus effect becomes more pronounced. However, even if the subject is off the focus plane, the circle is perceived as a single point if its radius remains below a certain threshold, known as the “acceptable CoC.” Thus, even when a CoC exists on the image sensor, the resulting image is perceived as all-in-focus if the CoC is smaller than this threshold.

Additionally, the size of the aperture is another key factor influencing the size

of the CoC. As illustrated in Fig. 4.2 (b), radiance emitted from a subject passes through the lens and the aperture. With a larger aperture, the radiance forms a larger CoC on the image sensor. With a smaller aperture, only a portion of the light passing through the lens reaches the sensor, resulting in a smaller CoC. Consequently, a smaller aperture produces CoCs smaller than acceptable CoC, leading to an all-in-focus image. However, a smaller aperture also reduces the amount of light reaching the image sensor during the same exposure time, requiring a longer exposure to collect sufficient light. Therefore, capturing an all-in-focus image necessitates (1) a small aperture size and (2) stability to prevent camera movement during the extended exposure time.

4.3 Method

4.3.1 CoCoGaussian Framework

Our goal is to represent a 3D scene using only defocused multi-view images, leveraging fundamental photographic principles to guide our method. We generate 3D Gaussians to capture a blurring kernel shaped as the CoC. The whole framework of CoCoGaussian is on Fig. 4.3. First, we compute the CoC diameter using the Gaussian depth, derived from the means of the given base 3D Gaussian set $\mathbf{G}_B$, which consists of N Gaussians, along with the camera position and aperture information (Sec. 4.3.2). Next, for the base Gaussian set $\mathbf{G}_B$, we generate M CoC Gaussian sets $\mathbf{G}_{\text{CoC}}$, resulting in a total of $(M \times N)$ Gaussians to capture the CoC shape. However, when the scene contains objects with refraction or reflection, the Gaussian

depth may be unreliable, which can lead to suboptimal CoC diameter. To address this issue, we propose an adaptive CoC Gaussian generation method that minimizes the dependence on depth for $\mathbf{G}_{\text{CoC}}$ (Sec. 4.3.3). Finally, we incorporate adaptive rotation quaternion and scaling vector adjustments, inspired by the Deblurring 3DGS [8] approach, to allow flexible learning of $\mathbf{G}_{\text{CoC}}$. Furthermore, we apply Eq. (3.7) to images rendered from $\mathbf{G}_B$ and $\mathbf{G}_{\text{CoC}}$, using a weighted sum approach [9,30] to create the final blurry image. This photography-prior-based approach allows us to accurately represent 3D scenes using only defocused images while adhering to realistic defocus principles.

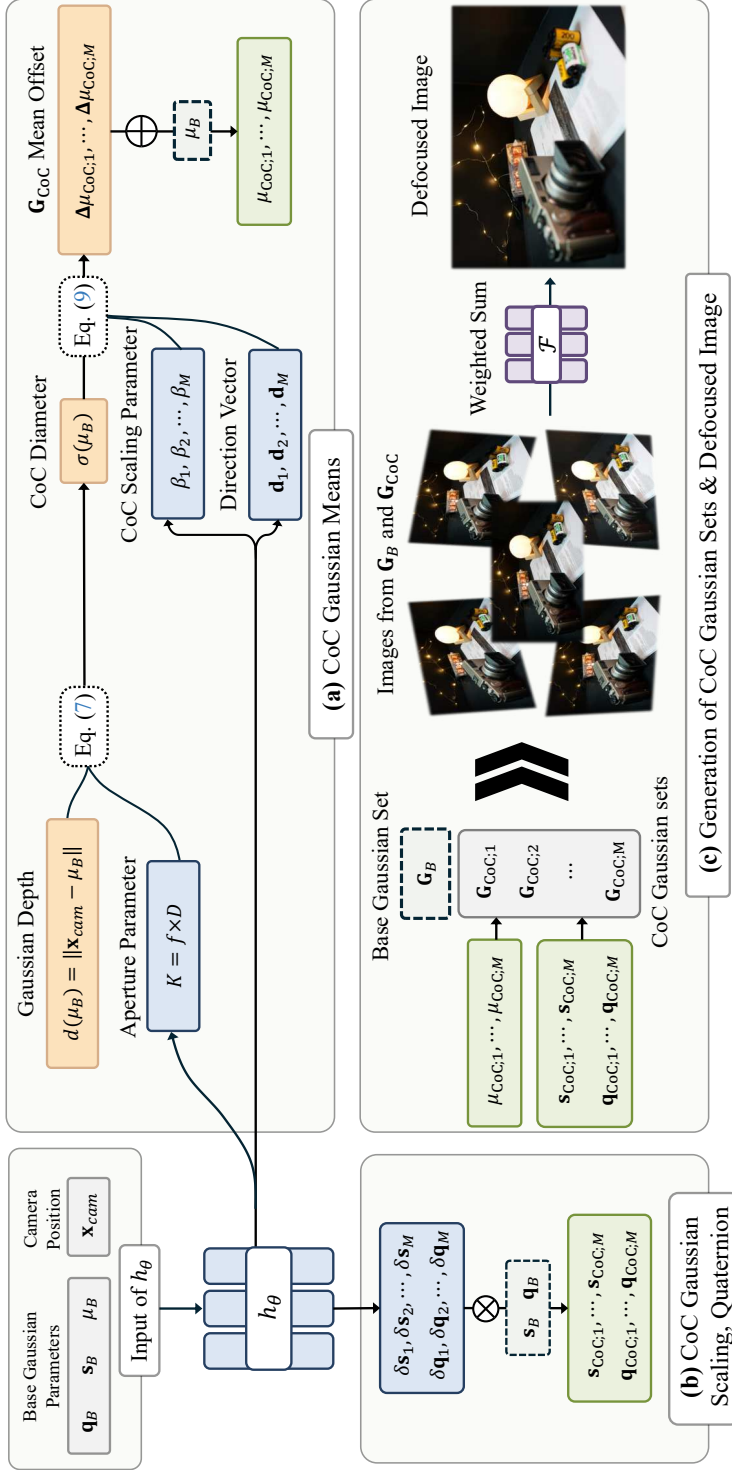


Figure 4.3: We take the camera position $\mathbf{x}_{cam}$ and the base Gaussian parameters μ_B , $\mathbf{s}_B$, and $\mathbf{q}_B$ as inputs to the MLP h_θ , which produces five outputs in total. (a) We set the depth $d(\mu_B)$ as the Euclidean distance between $\mathbf{x}_{cam}$ and μ_B . Using the output K from h_θ , the $d(\mu_B)$, and the learnable focus plane d_F , we apply Eq. (4.2) to determine the CoC diameter. The diameter, combined with the outputs β and $\mathbf{d}$ from h_θ , is then used in Eq. (4.4) to compute the offset values for μ_{CoC} . By adding these offsets to μ_B , we obtain the $\mathbf{G}_{CoC}$ means. (b) We obtain $\mathbf{s}_{CoC}$ and $\mathbf{q}_{CoC}$ by applying Eqs. (4.5) and (4.6) to $\delta \mathbf{s}_{CoC}$ and $\delta \mathbf{q}_{CoC}$, the outputs of h_θ . (c) Using μ_{CoC} , $\mathbf{s}_{CoC}$, and $\mathbf{q}_{CoC}$, we get the $\mathbf{G}_{CoC}$. Finally, we rasterize $\mathbf{G}_{CoC}$ along with $\mathbf{G}_B$ to produce $(M + 1)$ images, and apply a weighted sum to generate the final defocused image.

4.3.2 Circle of Confusion from 3D Gaussians

To model the CoC, we assume an ideal system that ignores distortions such as lens aberrations. Before generating the CoC Gaussian sets $\mathbf{G}_{\text{CoC}}$, we first need to obtain the CoC diameter, which depends on several factors: the aperture diameter D , focal length f , focus plane d_F , and depth p . The focus plane represents the distance from the camera position to the plane in focus. We define the depth $d(\mu_B)$ as the Euclidean distance between the camera position $\mathbf{x}_{cam} \in \mathbb{R}^3$ and the means of the base Gaussian set $\mu_B \in \mathbb{R}^{N \times 3}$. Exploiting these values, we express the CoC diameter $\sigma(\mu_B)$ as follows [26]:

$$\sigma(\mu_B) = fD \times \frac{|d(\mu_B) - d_F|}{d(\mu_B)(d_F - f)}, \quad (4.1)$$

where d_F is set to a learnable parameter and initialized as the average distance between $\mathbf{x}_{cam}$ and the points in the SfM point cloud to ensure stable training. Note that the focus plane d_F is specific to each input image. Additionally, because the focal length f is usually much smaller than the focus plane d_F , we modify Eq. (4.1) as follows:

$$\sigma(\mu_B) \approx K \times \left| \frac{1}{d(\mu_B)} - \frac{1}{d_F} \right|, \quad (4.2)$$

where the product of the focal length and aperture diameter is represented as a single learnable scalar $K = f \times D$ [6].

We design a simple MLP h_θ parameterized by θ to estimate the scalar K . Then, the aperture parameter K is obtained by passing these features as inputs to h_θ , along with the scaling vectors $\mathbf{s}_B \in \mathbb{R}^{N \times 3}$ and rotation quaternions $\mathbf{q}_B \in \mathbb{R}^{N \times 4}$ of $\mathbf{G}_B$.

This approach allows us to obtain the CoC diameter $\sigma(\mu_B)$ for the $\mathbf{G}_B$, marking the first step in modeling $\mathbf{G}_{\text{CoC}}$.

4.3.3 Adaptive CoC Gaussian Generation

After obtaining the CoC diameters, we use these diameters to generate M CoC Gaussian sets $\mathbf{G}_{\text{CoC}}$. For the Gaussian parameters of $\mathbf{G}_{\text{CoC}}$, we set the spherical harmonics (SH) coefficients and opacity values identical to those of $\mathbf{G}_B$, while varying only the means μ , scaling vectors $\mathbf{s}$, and rotation quaternions $\mathbf{q}$. The means of each CoC Gaussian set are created by adding offsets $\Delta\mu_{\text{CoC}} \in \mathbb{R}^{M \times N \times 3}$ to μ_B , defined as $\mu_{\text{CoC}} = \mu_B + \Delta\mu_{\text{CoC}}$. Since $\Delta\mu_{\text{CoC}}$ are 3D vector sets, we obtain unit vector sets $\mathbf{d} \in \mathbb{R}^{M \times N \times 3}$ representing the directions from $\mathbf{G}_B$ to $\mathbf{G}_{\text{CoC}}$, which are additional outputs from the MLP h_θ discussed in the previous section. Moreover, since $\mathbf{G}_{\text{CoC}}$ consist of a total of M Gaussian sets, The direction vectors $\mathbf{d}$ consist of M instances:

$$\Delta\mu_{\text{CoC};m} = \frac{\sigma(\mu_B)}{2} \mathbf{d}_m, \text{ where } m \in M. \quad (4.3)$$

However, theses offsets $\Delta\mu_{\text{CoC}}$ presents two potential issues. First, this approach places all $\mathbf{G}_{\text{CoC}}$ only on the boundary of the CoC, which limits its ability to fully capture the defocus effect. Modeling the entire CoC, including its interior, would allow for a more accurate and flexible representation of defocus blur. The second issue arises when there are refractive or reflective surfaces in the scene, as these subjects can lead to incorrect optimization of the Gaussian means, making the CoC diameter obtained through Eq. (4.2) unreliable. To address these problems, we introduce sim-

ple learnable CoC scaling parameters $\beta \in (0, 1]$, which are additional outputs of h_θ . The parameter β is learned within the range of 0 to 1, ensuring that μ_{CoC} resides within the CoC boundary. Additionally, even when depth is inaccurately measured and the predicted CoC diameter is larger than the actual size, β adaptively scales it down. Thus, the offset $\Delta\mu_{\text{CoC}}$ is modified by applying β as follows:

$$\Delta\mu_{\text{CoC};m} = \frac{\sigma(\mu_B)}{2}\beta_m\mathbf{d}_m. \quad (4.4)$$

Additionally, similar to Deblurring 3DGS [8], we introduce scaling factors $\delta\mathbf{s}_{\text{CoC}} \in \mathbb{R}^{M \times N \times 3}$ and $\delta\mathbf{q}_{\text{CoC}} \in \mathbb{R}^{M \times N \times 4}$, which allow flexible learning of the scaling vectors $\mathbf{s}_{\text{CoC}}$ and rotation quaternions $\mathbf{q}_{\text{CoC}}$ of $\mathbf{G}_{\text{CoC}}$. These factors are also the outputs of h_θ . The parameters $\delta\mathbf{s}_{\text{CoC}}$ and $\delta\mathbf{q}_{\text{CoC}}$ are multiplied by the corresponding parameters of the base Gaussian set $\mathbf{G}_B$, specifically $\mathbf{s}_B$ and $\mathbf{q}_B$, and are expressed as follows:

$$\mathbf{s}_{\text{CoC};m} = \mathbf{s}_B \times \delta\mathbf{s}_{\text{CoC};m}, \quad (4.5)$$

$$\mathbf{q}_{\text{CoC};m} = \mathbf{q}_B \times \delta\mathbf{q}_{\text{CoC};m}, \quad (4.6)$$

where the scaling parameters are constrained by the fixed values $\delta\mathbf{s}_{\text{max}}$ and $\delta\mathbf{q}_{\text{max}}$: $\delta\mathbf{s}_{\text{CoC}} \in [1, \delta\mathbf{s}_{\text{max}}]$ and $\delta\mathbf{q}_{\text{CoC}} \in [1, \delta\mathbf{q}_{\text{max}}]$.

We obtain a total of M means of CoC Gaussian sets μ_{CoC} , scaling vectors $\mathbf{s}_{\text{CoC}}$, and rotation quaternion $\mathbf{q}_{\text{CoC}}$. They are combined with the opacity and SH coeffi-

cients of $\mathbf{G}_B$, generate the M Gaussian sets $\mathbf{G}_{\text{CoC}}$:

$$\mathbf{x}_{\text{CoC};m}, \mathbf{s}_{\text{CoC};m}, \mathbf{q}_{\text{CoC};m}, \alpha_B, \text{SH}_B \rightarrow \mathbf{G}_{\text{CoC};m}. \quad (4.7)$$

We rasterize the resulting M Gaussian sets along with the base Gaussian set, producing $(M + 1)$ images. Then, using the weighted sum approach from Eq. (3.7), as detailed in the following section, we obtain the final blurry image.

4.3.4 Optimization

Weighted Sum. To apply Eq. (3.7) to the $(M + 1)$ images $\mathbf{I}$ rasterized from $\mathbf{G}_B$ and $\mathbf{G}_{\text{CoC}}$, we adopt the methods of recent studies [9,30]. We use a shallow CNN $\mathcal{F}$ to compute pixel-wise weights $\mathcal{W}$ for these images, applying a softmax function to ensure that the weights for each pixel sum to 1. Then, we multiply each image by its corresponding weight and sum the results to obtain the final defocused image $\mathcal{I}_{\text{blur}}$:

$$\mathcal{I}_{\text{blur}} = \sum_{m=1}^{M+1} \mathcal{I}_m \mathcal{W}_m, \text{ where } \mathcal{W} = \text{softmax}(\mathcal{F}(\mathbf{I})), \quad (4.8)$$

where $\mathcal{I}_m$ represents the image rasterized from $\mathbf{G}_{\text{CoC};m}$, and $\mathcal{I}_{M+1}$ represents the image rasterized from $\mathbf{G}_B$.

Objective. We minimize the $\mathcal{L}_1$ loss and D-SSIM loss $\mathcal{L}_{\text{D-SSIM}}$ between the ground truth defocused image and the output defocused image. The $\mathcal{L}_1$ loss reduces the pixel-wise differences between images, while the D-SSIM loss minimizes structural

differences between them. The final objective $\mathcal{L}_{rgb}$ is defined as follows:

$$\mathcal{L}_{rgb} = (1 - \lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{D-SSIM}, \quad (4.9)$$

where we use $\lambda = 0.3$ for all experiments.

4.4 Experiments

Datasets. We evaluate our method on three different datasets: the Deblur-NeRF [1] synthetic dataset, the Deblur-NeRF real-world dataset, and the DoF-NeRF [6] real-world dataset. The Deblur-NeRF synthetic dataset consists of 5 scenes, each containing defocused images generated using the built-in function of Blender [31]. The Deblur-NeRF real-world dataset includes 10 scenes, with images captured by enlarging the aperture on a Canon EOS RP. The DoF-NeRF real-world dataset comprises 7 scenes, each containing 20 to 30 image triplets. Each triplet includes an all-in-focus image, along with two defocused images focused on the background and foreground. Since DoF-NeRF does not provide training code, we use half of the training defocused images with a foreground focus and the other half with a background focus. All camera poses and initial point clouds are obtained through COLMAP [32,33].

Implementation Details CoCoGaussian is built upon 3DGS [7] and Deblurring 3DGS [8], trained with a total of 30k iterations with the number of CoC Gaussians, M , set to 5. For coarse geometry in the early training stages, h_θ is not trained during the first 2k iterations and begins training afterward. We set $\delta\mathbf{q}_{\max}$ and $\delta\mathbf{s}_{\max}$

to 1.1. Additionally, after h_θ has been coarsely trained for 4k iterations, CNN $\mathcal{F}$ starts training. Note that, prior to the training of $\mathcal{F}$, the $(M + 1)$ output images are averaged to obtain a blurry image. Additionally, we apply a positional encoding layer [4], γ , to 3D points (*i.e.*, $\mathbf{x}_{cam}$ and μ_B):

$$\gamma(\mathbf{x}_{cam}) = \left(\sin(2^k \pi \mathbf{x}_{cam}), \cos(2^k \pi \mathbf{x}_{cam}) \right)_{k=0}^{L-1}, \quad (4.10)$$

$$\gamma(\mu_B) = \left(\sin(2^k \pi \mu_B), \cos(2^k \pi \mu_B) \right)_{k=0}^{L-1}, \quad (4.11)$$

where L denotes the number of the frequencies. The h_θ consists of 3 serial MLP layers, each with 64 hidden units, and parallelized 4 head layers for K , β , $\mathbf{d}$, and $\delta(\mathbf{q}, \mathbf{s})$. The CNN $\mathcal{F}$ comprises 4 convolutional layers with 64 channels each. To compensate for the sparse initial point cloud, we adopt the approach from Deblurring 3DGS, adding approximately 200k additional points after 2.5k iterations and pruning Gaussians based on depth. All experiments are conducted on either an NVIDIA RTX 3090 or NVIDIA V100 GPU.

4.4.1 Rendering Results

Quantitative Results. We quantitatively evaluate our CoCoGaussian using the following three metrics: peak signal-to-noise ratio (PSNR), structural similarity index measure (SSIM) [35], and learned perceptual image patch similarity (LPIPS) [36]. We compare our approach to several state-of-the-art methods, including both ray-tracing [1,10,22] and rasterization-based methods [8,9]. Quantitative results are shown

Table 4.1: Comparisons on Deblur-NeRF synthetic and real-world scene dataset. We evaluate the performance on three metrics (PSNR, SSIM, LPIPS). “*” denotes the results obtained by reproducing the released code. The orange and yellow cells respectively indicate the highest and second-highest value.

Methods	Synthetic Scene [1]			Real-World Scene [1]		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Naive NeRF [4]	25.95	0.7791	0.2303	22.40	0.6661	0.2310
3DGS [7]	25.11	0.7476	0.2148	23.38	0.6655	0.3140
Restormer [34] + 3DGS	-	-	-	23.14	0.6669	0.2136
Deblur-NeRF [1]	28.37	0.8527	0.1188	23.47	0.7199	0.1207
PDRF-10 [22]	30.08	0.8931	0.1101	23.85	0.7382	0.1746
DP-NeRF [10]	29.33	0.8713	0.0987	23.67	0.7299	0.1082
Deblurring 3DGS* [8]	28.90	0.8912	0.1052	23.54	0.7383	0.1232
BAGS* [9]	30.65	0.9128	0.0631	23.48	0.7408	0.0962
Ours	30.84	0.9212	0.0478	23.70	0.7531	0.0825

in Tab. 4.1 and Tab. 4.2, where our approach achieves the best performance across all metrics except for PSNR on the Deblur-NeRF real-world dataset. Note that we report the reproduced performance scores for Deblurring 3DGS [8] and BAGS [9] using their official codes. The Deblur-NeRF real-world dataset inherently presents challenges, as there are illumination differences between the sharp and defocused images, leading to relatively lower PSNR score. We also observe that the two-stage baseline that first applies Restormer for image deblurring and then trains 3DGS obtains PSNR scores comparable to those of our method in some cases. However, it shows a much larger gap in LPIPS, indicating that image-space deblurring may recover pixel-wise fidelity to some extent but fails to preserve perceptually consistent

Table 4.2: Comparisons on DoF-NeRF real-world scene dataset.

Methods	DoF-NeRF Real-World Scene [6]		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGS [7]	25.72	0.8291	0.1817
DP-NeRF [10]	26.80	0.8145	0.1790
Deblurring 3DGS [8]	26.58	0.8386	0.1708
BAGS [9]	29.87	0.8816	0.1100
Ours	30.14	0.9127	0.0701

details and view-consistent 3D appearance. For the DoF-NeRF dataset, CoCoGaussian achieves outstanding performance across all metrics, with an approximately 0.04 reduction in LPIPS compared to BAGS. Since the DoF-NeRF dataset has a higher resolution than the Deblur-NeRF dataset, this performance drop in BAGS is likely due to its requirement for manual adjustment of blurring kernel size, highlighting an intrinsic limitation. Conversely, Deblurring 3DGS shows minimal improvement over 3DGS, which can also be attributed to its limited kernel size relative to the higher resolution.

Qualitative Results. For qualitative evaluation, we present the rendering results of 3DGS [7], Deblurring 3DGS [8], and BAGS [9] in Fig. 4.4. Our method demonstrates higher-fidelity results compared to others. Specifically, in the fourth row of our results, the white line on the rightside is rendered more sharply, and in the fifth row, the detailed quality of the graphic card appears superior to that of BAGS.

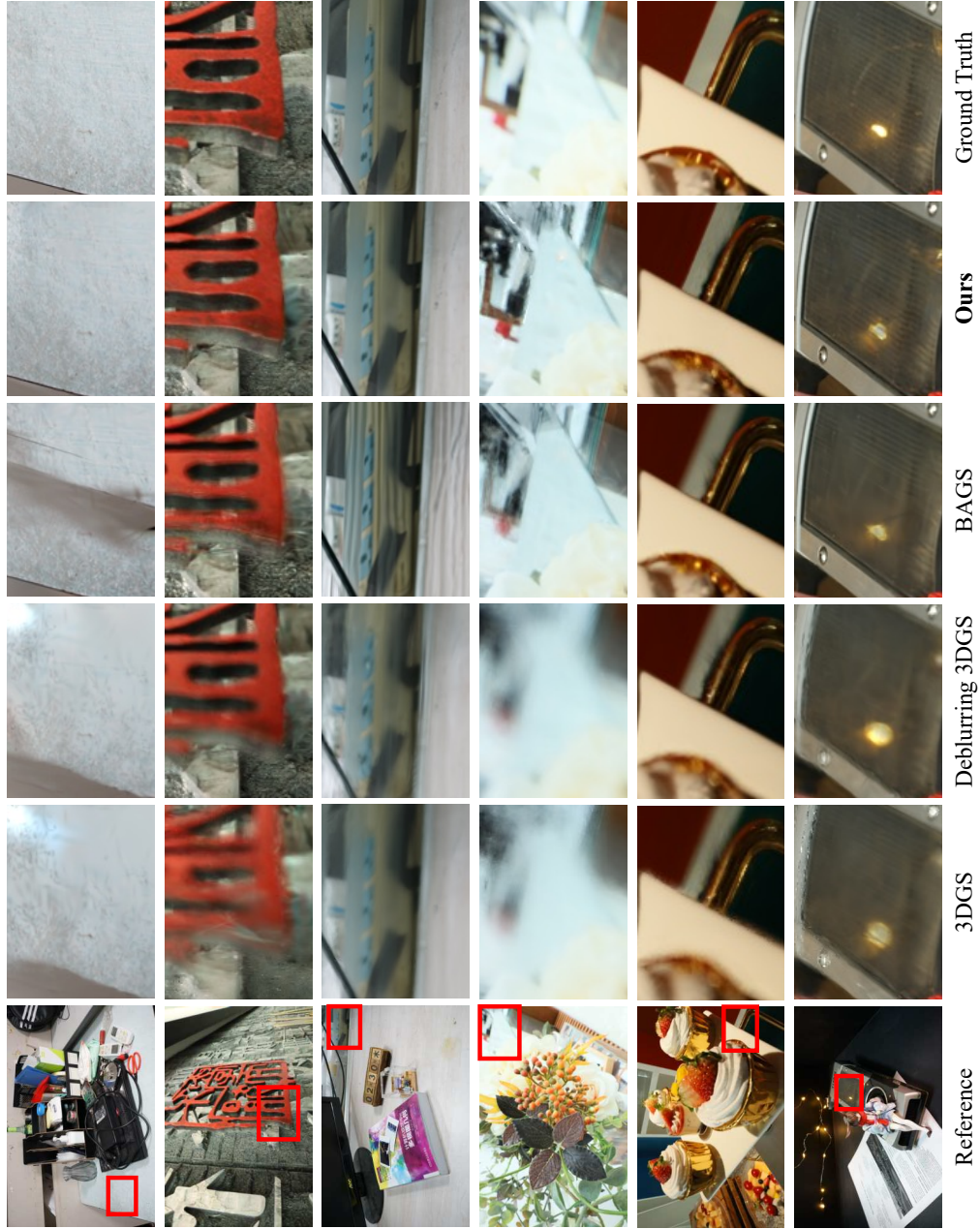


Figure 4.4: Qualitative comparison on the Deblur-NeRF and DoF-NeRF datasets.

4.4.2 Ablation Study

To validate the effectiveness of our proposed methods, we perform a series of ablation studies, with all experiments conducted on the DoF-NeRF real-world dataset. We chose this dataset as it is based on real captured images and does not present the illumination issues found in the Deblur-NeRF real-world dataset, ensuring more reliable evaluation. The results are summarized in Tab. 4.3, where the baseline denotes naive 3DGS [7].

Circle of Confusion. To demonstrate the impact of our core concept, CoC, we conduct an experiment that excludes the CoC from the model, retaining only β_m and $\mathbf{d}_m$. This configuration yields significantly lower performance than the full model, indicating an over-reliance on parametric learning. The result suggests that without the CoC, which leverages depth and aperture information, the means of the generated Gaussian sets are not optimally arranged, leading to overfitting.

CoC Direction Vector. In this experiment, we retain the CoC concept but exclude the learnable direction vectors $\mathbf{d}_m$. Instead of using learnable vectors, we use fixed direction vectors, arranging M CoC Gaussians evenly in a circular pattern on a plane perpendicular to the vector between the camera position and the base Gaussian. This configuration slightly underperforms compared to the full model, as the fixed directions restrict optimal Gaussian position, preventing adaptation to the most reliable configuration.

Table 4.3: Ablation Results of CoCoGaussian.

Methods		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Baseline [7]		25.72	0.8291	0.1817
w/o CoC		28.29	0.8778	0.0927
w/o CoC direction vector		28.91	0.8893	0.0896
w/o CoC scaling		29.46	0.9058	0.0793
w/o aperture parameter		27.37	0.8510	0.1113
Ours Full		30.14	0.9127	0.0701

Aperture Parameter. The aperture parameter K includes the aperture diameter and focal length, and it plays a critical role in determining the size of the CoC. Without this parameter, there is an upper limit to the CoC diameter calculation, making accurate computation challenging. Experimental results indicate that although performance is somewhat lower than the full model, it is improved over the baseline. This suggests that while the model is unable to determine precise CoC size without K , it still learns the focus plane and captures the in-focus region accurately.

CoC Scaling Factor. In this setup, we include the learnable direction vector while omitting the CoC scaling factor β_m . Although the performance is slightly better than when the direction vector is excluded, it remains lower than that of the full model. Without the CoC scaling factor, CoC Gaussian generation becomes overly dependent on depth, leading to errors in CoC diameter calculation when depth values are inaccurate. Including the CoC scaling factor enables the full model to handle uncertain depth more robustly, improving performance. We visualize scenes with reflections and refractions in Fig. 4.5. The top scene models a transparent cup

where light refracts, and the rendered result without CoC scaling shows significant artifacts on the cup. Similarly, in the bottom scene, where a metallic pipe causes light reflection, the absence of CoC scaling leads to many floaters in the affected areas. This indicates that the CoC Gaussians without CoC scaling factor are overfitted to the training images, resulting in poorly optimized base Gaussian positions μ_B . Furthermore, this overfitting may potentially affect the covariance of the Gaussians and their SH coefficients. In other words, objects with reflections or refractions often exhibit inconsistent radiance depending on the view direction, making it challenging for 3DGS [7] to properly optimize for such textures. This inherent limitation of 3DGS causes the CoC Gaussians to overly rely on inaccurately optimized depth, leading to suboptimal outputs. By incorporating the CoC scaling factor during training, we enable robust modeling of CoC Gaussians even in scenes with challenging reflective or refractive surfaces.

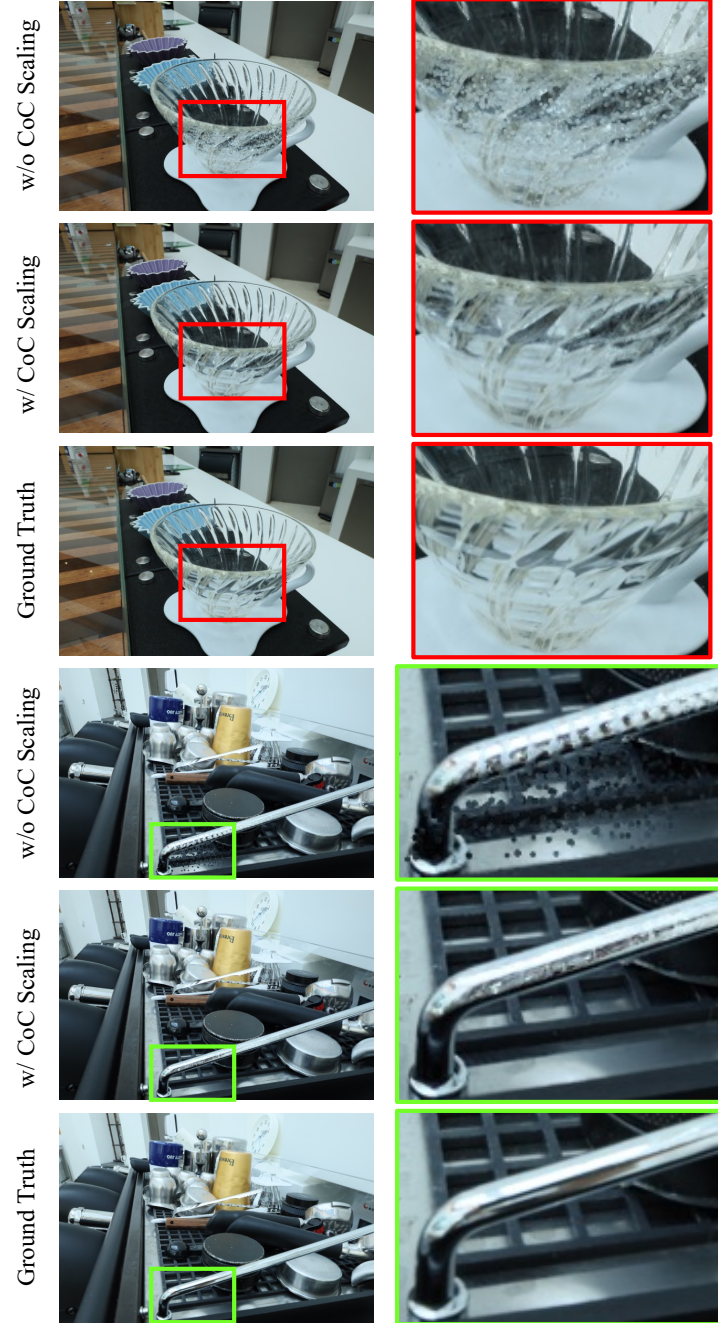


Figure 4.5: Qualitative Ablation of CoC Scaling Factor. The figures are from CUP and TOOLS scenes of Deblur-NeRF [1] real-world dataset.

Number of CoC Gaussians. We conduct ablative experiments on the number of CoC Gaussians, M , with the results as shown in Tab. 4.4. For PSNR and SSIM, the scores vary inconsistently as M changes. This inconsistency arises primarily from differences in light exposure between defocused images for training and sharp images for evaluation, which depends on the aperture size and exposure time used to equalize the amount of light. In the DoF-NeRF [6] dataset, defocused images are captured with an aperture of $f/4$ and an exposure time of $1/13$ seconds, while sharp images are captured with an aperture of $f/11$ and an exposure time of 0.8 seconds. The change from $f/4$ to $f/11$ reduces light by a factor of $1/8$ due to a 3-stop aperture decrease. Meanwhile, the exposure time increases by a factor of approximately 10.4, from $1/13$ seconds to 0.8 seconds. Accounting for both aperture and exposure time, sharp images receive 1.3 times more light than defocused images, making the latter slightly darker. Consequently, higher PSNR and SSIM scores do not necessarily indicate better quality.

On the other hand, for LPIPS, larger values of M generally result in better performance. Since LPIPS evaluates quality based on high-level features, such as geometric differences, rather than pixel-level or luminance-based differences, it aligns more closely with human visual perception. As a result, a lower LPIPS score is a more reliable measure of image quality compared to higher PSNR or SSIM scores.

Table 4.4: Quantitative Results for the Number of CoC Gaussian Sets M . The orange and yellow cells respectively indicate the highest and second-highest values.

Methods	AMIYA			Book			CAMERA			Desk		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
$M = 2$	30.06	0.9313	0.0915	30.74	0.9277	0.0626	29.44	0.9276	0.0725	30.07	0.9163	0.0652
$M = 3$	31.02	0.9389	0.0882	31.89	0.9377	0.0576	29.36	0.9270	0.0723	30.57	0.9273	0.0592
$M = 4$	30.71	0.9395	0.0858	30.45	0.9207	0.0600	29.63	0.9283	0.0707	29.68	0.9193	0.0532
$M = 5$	30.59	0.9406	0.0867	31.58	0.9304	0.0585	30.21	0.9372	0.0609	29.94	0.9188	0.0559
$M = 6$	31.21	0.9411	0.0829	30.94	0.9255	0.0573	29.39	0.9266	0.0698	29.48	0.9151	0.0524
Average												
Methods	KENDO			PLANT			SHELF			Average		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
$M = 2$	25.77	0.8414	0.1303	31.21	0.8899	0.0901	32.02	0.9351	0.0455	29.90	0.9099	0.0797
$M = 3$	25.83	0.8441	0.1264	30.78	0.8814	0.0859	29.91	0.9131	0.0462	29.91	0.9099	0.0765
$M = 4$	25.76	0.8246	0.1277	29.59	0.8642	0.0830	31.20	0.9245	0.0480	29.57	0.9030	0.0755
$M = 5$	26.26	0.8560	0.1160	31.01	0.8881	0.0693	31.38	0.9181	0.0431	30.14	0.9127	0.0701
$M = 6$	26.67	0.8607	0.1117	27.58	0.8661	0.0894	31.42	0.9243	0.0460	29.53	0.9085	0.0728

Table 4.5: Results on NeRF-LLFF [4,5] Dataset.

Methods		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGS [7]		27.10	0.8619	0.0569
Ours		27.76	0.8738	0.0372

4.4.3 Experiments on All-in-Focus Images.

We conduct novel view synthesis experiments using all-in-focus images to verify the generalization performance of CoCoGaussian. We utilize the NeRF-LLFF [4,5] dataset, which contains real-world images. As shown in Tab. 4.5, our approach outperforms the baseline 3DGS [7] across all metrics. In practice, when the aperture is very small, radiance from a point on the subject forms a tiny CoC on the image sensor after passing through the aperture, even if the subject is slightly away from the focus plane. Because the CoC is so small, the resulting image is not perceived as defocused. By modeling this small circle, which is difficult to detect with the human eye, our approach achieves higher performance than the baseline 3DGS.

4.4.4 Customizable Depth of Field and Focus Plane

Customizing the depth of field and focus settings is essential in various applications, such as augmented reality (AR) and virtual reality (VR), where control over depth cues and focus effects can greatly enhance visual realism and viewer engagement. Since our approach incorporates the aperture parameter K in CoC modeling, it enables customization of both CoC size and depth of field by adjusting this parameter. The top row in Fig. 4.6 visualizes increasing depth of field by gradually reducing K from left to right. As the focus plane is positioned at a shallow depth, the model consistently captures focus in this region. However, in the leftmost image, the large aperture causes the CoC size to increase at deeper depths, resulting in defocus blur.

Additionally, our model can customize the focus plane of defocused images, as shown in the bottom row of Fig. 4.6, where the focus plane shifts from shallow depth on the left to deeper depth on the right. The shallow regions are in focus on the leftmost image, while deeper areas remain out of focus, and the opposite holds for the rightmost image. This level of control over focus and depth effects is essential for achieving realistic depth cues. Therefore, CoCoGaussian not only demonstrates superior capability in synthesizing sharp novel view images but also provides enhanced flexibility in adjusting aperture and focus plane settings.

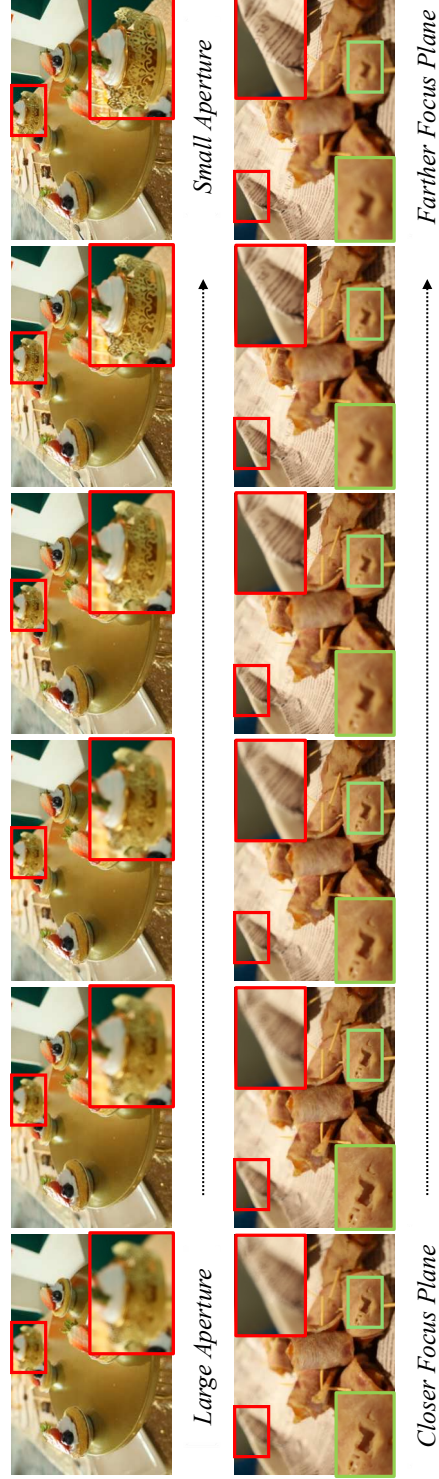


Figure 4.6: Visualization of Aperture parameter and Focus Plane Customization. The top row of images decreases the aperture parameter K from left to right, while the bottom row moves the focus plane d_F further from the camera from left to right.

Table 4.6: Computational Efficiency and Speed. * indicates that the rendering speed is identical to that of the corresponding model.

Methods	GPU Memory (GB)	Training Time (min)	Rendering Speed
BAGS [9]	4.7	47	*Mip-Splatting [2]
Ours	5.3	45	*3DGS [7]

4.4.5 Computational Efficiency and Speed

We compare our GPU usage, training time, and rendering speed with BAGS [9], a state-of-the-art method, on the Deblur-NeRF real-world dataset using an NVIDIA RTX 3090. As shown in the Tab. 4.6, CoCoGaussian achieves comparable resource consumption and training time while delivering superior performance.

After the training phase, CoCoGaussian renders sharp images using only the $\mathbf{G}_B$ through a naive 3DGS. In the other words, the rendering speed and memory cost are the same as 3DGS alone. Therefore, our method is feasible for real-time applications that rely on 3DGS rendering speeds.

4.4.6 CoC Visualization

We visualize the CoC for various types of images in Fig. 4.7 and Fig. 4.8. To simplify the visualization, we randomly sample a subset of positions from numerous Gaussians. The points in Fig. 4.7 represent the positions of Gaussians for defocused images. For images where the focus plane is close to the camera, CoCoGaussian generates CoC sizes that are very small for shallow depths and progressively larger for deeper depths. However, when the focus plane is farther from the camera, the CoC sizes reconstructed at greater depths are smaller. This demonstrates the effectiveness of our modeling, accurately reflecting the principles of defocus blur.

In contrast, Fig. 4.8 visualizes the CoC for an all-in-focus scene [4], where the CoC sizes remain uniformly small regardless of depth. This suggests that the learned aperture size is very small, ensuring precise modeling of all-in-focus images. In other words, CoCoGaussian can model small apertures and capture subtle defocus effects that are imperceptible to the human eye, which contributes to its superior performance compared to naive 3DGS [7], as demonstrated in Tab. 4.4 of the main paper. In summary, our model can accurately represent 3D scenes for both defocused and all-in-focus images, highlighting its versatility and robustness.



Figure 4.7: Gaussian Positions of Defocused Images. The black and gray dots indicate base and CoC Gaussians, respectively.

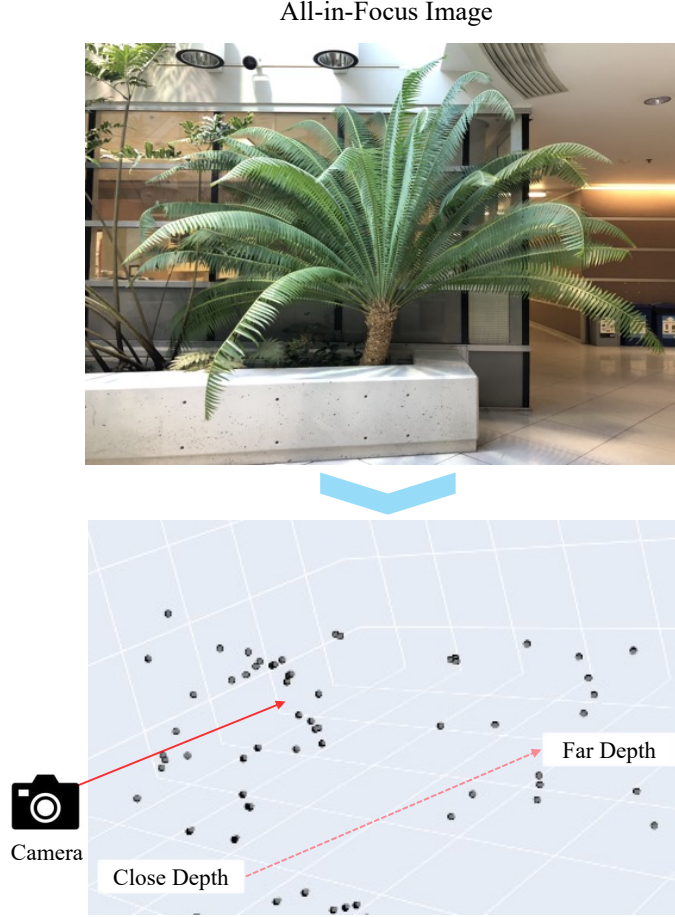


Figure 4.8: Gaussian Positions of All-in-Focus Images.

4.4.7 Per-Scene Quantitative Results

We present the performance for individual scenes across all datasets in Tabs. 4.7 to 4.9. CoCoGaussian achieves the best LPIPS scores in all scenes except for the CORAL scene in the Deblur-NeRF Real-World dataset. As discussed in Sec. 4.4.2, this indicates that CoCoGaussian exhibits high-level feature representation.

Table 4.7: Per-Scene Quantitative Results on Deblur-NeRF [1] Synthetic Dataset.

Methods	FACTORY				COZYROOM				POOL				TANABATA				TROLLEY			
	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓
NeRF [4]	25.36	0.7847	0.2351	0.2351	30.03	0.8926	0.0885	0.0885	27.77	0.7266	0.3340	0.3340	23.90	0.7811	0.2142	0.2142	22.67	0.7103	0.2799	0.2799
3DGS [7]	24.52	0.8057	0.1842	0.1842	30.09	0.9024	0.0692	0.0692	20.14	0.4451	0.5094	0.5094	23.08	0.7981	0.1710	0.1710	22.26	0.7400	0.2281	0.2281
Deblur-NeRF [1]	28.03	0.8628	0.1127	0.1127	31.85	0.9175	0.0481	0.0481	30.52	0.8246	0.1901	0.1901	26.26	0.8517	0.0995	0.0995	25.18	0.8067	0.1436	0.1436
PDRF-10 [22]	30.90	0.9138	0.1066	0.1066	32.29	0.9305	0.0518	0.0518	30.97	0.8408	0.1893	0.1893	28.18	0.9006	0.0819	0.0819	28.07	0.8799	0.1210	0.1210
DP-NeRF [10]	29.26	0.8793	0.1035	0.1035	32.11	0.9215	0.0386	0.0386	31.44	0.8529	0.1563	0.1563	27.05	0.8635	0.0779	0.0779	26.79	0.8395	0.1170	0.1170
Deblurring 3DGS [8]	27.39	0.8922	0.1160	0.1160	31.29	0.9201	0.0505	0.0505	31.27	0.8565	0.1556	0.1556	27.04	0.9029	0.0872	0.0872	27.53	0.8843	0.1167	0.1167
BAGS [9]	30.87	0.9334	0.0724	0.0724	32.45	0.9312	0.0289	0.0289	31.78	0.8645	0.0932	0.0932	29.19	0.9278	0.0405	0.0405	28.97	0.9070	0.0804	0.0804
Ours	30.15	0.9300	0.0489	0.0489	33.02	0.9410	0.0213	0.0213	31.96	0.8788	0.0803	0.0803	29.65	0.9396	0.0263	0.0263	29.41	0.9165	0.0620	0.0620

Table 4.8: Per-Scene Quantitative Results on Deblur-NeRF [1] Real-World Dataset.

Methods	CAKE			CAPS			CISCO			CORAL			CUPCAKE		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
NeRF [4]	24.42	0.7210	0.2250	22.73	0.6312	0.2801	20.72	0.7217	0.1256	19.81	0.5658	0.2155	21.88	0.6809	0.2689
3DGS [7]	20.16	0.5903	0.2082	19.08	0.4355	0.4329	20.01	0.6931	0.1781	19.50	0.5519	0.3111	21.53	0.6794	0.2081
Deblur-NeRF [1]	26.27	0.7800	0.1282	23.87	0.7128	0.1612	20.83	0.7270	0.0868	19.85	0.5999	0.1160	22.26	0.7219	0.1214
PDRF-10 [22]	27.06	0.8032	0.1622	24.06	0.7102	0.2854	20.68	0.7239	0.0943	19.61	0.5894	0.2335	22.95	0.7421	0.1862
DP-NeRF [10]	26.16	0.7781	0.1267	23.95	0.7122	0.1430	20.73	0.7260	0.0840	20.11	0.6107	0.0960	22.80	0.7409	0.1178
Deblurring 3DGS [8]	26.91	0.8039	0.1136	24.45	0.7391	0.1509	20.55	0.7227	0.0816	18.99	0.5534	0.2767	22.11	0.7356	0.1021
BAGS [9]	26.53	0.7996	0.1113	24.15	0.7422	0.1391	20.31	0.7230	0.0759	19.63	0.6016	0.1114	21.52	0.6971	0.1214
Ours	26.65	0.8043	0.1037	24.62	0.7472	0.1334	20.83	0.7359	0.0680	19.57	0.5925	0.1105	22.48	0.7515	0.0739
Methods	CUPS			DAISY			SAUSAGE			SEAL			TOOLS		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
NeRF [4]	25.02	0.7581	0.2315	22.74	0.6203	0.2621	17.79	0.4830	0.2789	22.79	0.6267	0.2680	26.08	0.8523	0.1547
3DGS [7]	20.55	0.6459	0.3211	20.96	0.6004	0.2629	17.83	0.4718	0.2855	22.25	0.5905	0.3057	23.82	0.8050	0.1953
Deblur-NeRF [1]	26.21	0.7987	0.1271	23.52	0.6870	0.1208	18.01	0.4998	0.1796	26.04	0.7773	0.1048	27.81	0.8949	0.0610
PDRF-10 [22]	26.39	0.8066	0.1370	24.49	0.7426	0.1024	18.94	0.5686	0.2126	26.36	0.7959	0.1927	28.00	0.8995	0.1395
DP-NeRF [10]	26.75	0.8136	0.1035	23.79	0.6971	0.1075	18.35	0.5443	0.1473	25.95	0.7779	0.1026	28.07	0.8980	0.0539
Deblurring 3DGS [8]	26.23	0.8230	0.1014	23.39	0.7288	0.0979	18.83	0.5609	0.1470	26.04	0.8087	0.0988	27.86	0.9069	0.0619
BAGS [9]	26.14	0.8194	0.0901	23.00	0.7332	0.0540	18.66	0.5721	0.1176	26.16	0.8050	0.0967	28.72	0.9148	0.0450
Ours	26.20	0.8317	0.0773	23.39	0.7425	0.0503	19.20	0.5864	0.1007	26.10	0.8243	0.0663	27.91	0.9149	0.0416

Table 4.9: Per-Scene Quantitative Results on DoF-NeRF [6] Real-World Dataset.

Methods	AMIYA			Book			CAMERA			DESK		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
3DGS [7]	26.16	0.8718	0.1761	24.73	0.8480	0.1980	25.05	0.8429	0.1641	27.47	0.8760	0.1422
DP-NeRF [1]	28.64	0.8849	0.1421	28.51	0.8520	0.1669	26.50	0.8667	0.1332	27.57	0.8238	0.1712
Deblurring 3D-GS [22]	26.21	0.8728	0.1815	27.29	0.8754	0.1584	25.63	0.8486	0.1796	29.08	0.8478	0.1672
BAGS [10]	31.86	0.9453	0.0874	29.41	0.7515	0.2214	29.20	0.9248	0.0730	29.77	0.9175	0.0722
Ours	30.59	0.9406	0.0867	31.58	0.9304	0.0585	30.21	0.9372	0.0609	29.94	0.9188	0.0559
Methods	KENDO			PLANT			SHELF			AVERAGE		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
3DGS [7]	20.63	0.6839	0.2715	26.64	0.7897	0.2047	29.37	0.8913	0.1150	25.72	0.8291	0.1817
DP-NeRF [1]	19.64	0.6166	0.3205	28.07	0.8158	0.1668	28.64	0.8415	0.1525	26.80	0.8145	0.1790
Deblurring 3D-GS [22]	22.74	0.7559	0.1904	26.66	0.7900	0.2081	28.44	0.8798	0.1102	26.58	0.8386	0.1708
BAGS [10]	26.19	0.8457	0.1291	30.66	0.8644	0.1128	32.03	0.9223	0.0741	29.87	0.8816	0.1100
Ours	26.26	0.8560	0.1160	31.01	0.8881	0.0693	31.38	0.9181	0.0431	30.14	0.9127	0.0701

Chapter 5

Continuous Motion-Aware Gaussian Splatting from Motion-Blurred Images

5.1 Introduction

Novel view synthesis has recently garnered significant attention, with neural radiance fields (NeRF) [4] making considerable advancements in photo-realistic neural rendering. NeRF takes sharp 2D images from multiple views as input to reconstruct precise 3D scenes, which are crucial for applications such as augmented reality (AR) and virtual reality (VR). However, inefficient memory usage of NeRF’s volume rendering process poses challenges for real-time applications. To address this, 3D Gaussian Splatting (3DGS) [7] has recently emerged, offering an alternative by explicitly representing 3D scenes and enabling real-time rendering through a differentiable splatting method.

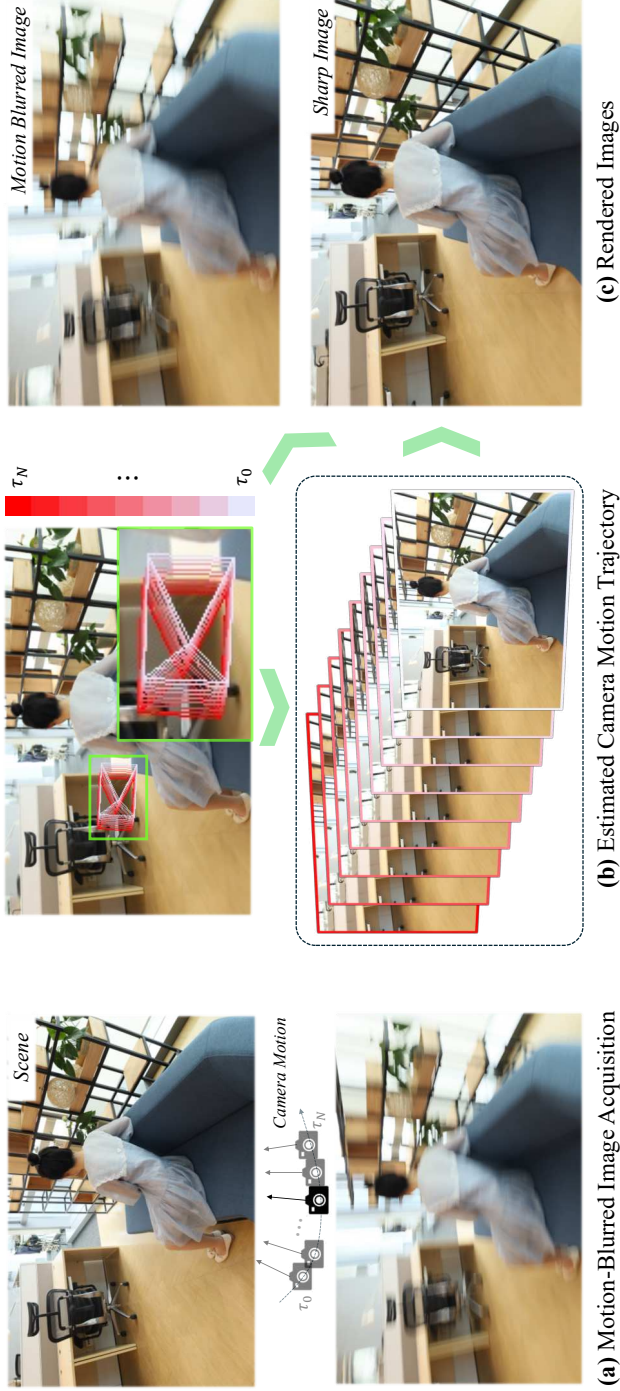


Figure 5.1: We propose CoMoGaussian, a novel framework for reconstructing 3D scenes from camera motion-blurred images. **(a)** Camera motion-blurred images are generated by the continuous movement of the camera during the exposure time. **(b)** The top image represents the camera poses obtained through CoMoGaussian, showing a continuous trajectory indicated by gradual change of colors. The bottom images are the rendering outputs from each camera pose. **(c)** By aggregating the rendered images obtained in step (b), we get the motion-blurred output image shown at the top, while the sharp output image is rendered from the given calibrated camera pose.

However, for precise 3D scene representation in real-world scenarios, it is essential to address various forms of image quality degradation, such as camera motion blur and defocus blur. Current methods like NeRF and 3DGS rely on sharp images as input, which assumes highly ideal conditions. Obtaining sharp images requires a large depth of field (DoF), necessitating a very small aperture setting [26]. However, a smaller aperture limits light intake, leading to longer exposure times. With longer exposure, even slight camera movement introduces complex motion blur in the images. Therefore, developing techniques that can handle motion blur in input images is crucial for accurate 3D scene reconstruction. Our research tackles this challenge by focusing on reconstructing precise 3D scenes from motion-blurred images, thus broadening the applicability of neural rendering in more realistic conditions.

Recently, several methods have been proposed to achieve sharp novel view rendering from camera motion-blurred images. Inspired by traditional blind image deblurring techniques, Deblur-NeRF [1] firstly introduces a method for deblurring 3D scenes. This approach designs a learnable kernel that intentionally blurs images during training, and during rendering, excludes the learned blurring kernel to render sharp novel view images. Following this approach, various methods targeting camera motion blur [8–10,22,23,25] have emerged, aiming to improve blurring kernel estimation accuracy. However, all of these methods predict the camera trajectory without enforcing continuity of camera motion throughout the exposure time (*e.g.*, simple spline functions [23,25]). This oversight is critical because motion blur arises from the continuous integration of scene radiance over the camera’s actual movement

during the exposure interval. When continuity is not properly accounted for, the predicted motion can exhibit abrupt or piecewise transitions that deviate from physically plausible camera trajectories, resulting in inaccurate blur approximation and suboptimal deblurring performance. While these methods can model simpler motion blur scenarios, they fail to capture complex, smoothly varying camera motion due to their lack of a temporally coherent representation.

In this paper, we propose a **C**ontinuous **M**otion-Aware **G**aussian Splatting, namely CoMoGaussian, a novel approach with three key contributions. First, we apply neural ordinary differential equations (ODEs) [11] to model continuous camera movement during exposure time, as illustrated in Fig. 5.1. By continuously modeling the camera trajectory in 3D space, we introduce a model that is fundamentally different from existing methods [1,10,22,23]. Second, we continuously model rigid body transformation over time to accurately capture the shape and size of the static subject throughout the camera movement. By leveraging a continuous representation of rigid motion, our approach better accounts for subtle variations in trajectory, leading to more precise reconstruction of static structures. Third, we introduce a continuous motion refinement (CMR) transformation, which enhances rigid motion modeling by incorporating learnable transformations, enabling more accurate approximation of motion blur trajectories. To integrate these components into a high-quality real-time rendering framework, we adopt Mip-Splatting [2], a differentiable rasterization-based approach built on 3DGS [7]. We evaluate and compare our approach on Deblur-NeRF [1] and ExbluRF [3] datasets, achieving state-of-the-art

results across the benchmarks. To demonstrate the effectiveness of CoMoGaussian, we conduct various ablative experiments on the proposed contributions.

5.2 Preliminaries

5.2.1 Neural Ordinary Differential Equations

Neural ODEs [11] are first proposed as an approach that interprets neural networks as the derivatives of a ODE systems, where ODEs represent the dynamics inherent to the hidden states. Specifically, neural ODEs are utilized to represent parameterized, time-continuous dynamics in the latent space, providing a unique solution given an initial value and numerical differential equation solvers [37]. Recently, there has been extensive research on neural ODEs. For instance, Latent-ODE [38] models the continuous dynamics of irregularly sampled time-series data, while Vid-ODE [39] models a smooth latent space by capturing the underlying continuous dynamics of videos.

Neural ODEs model a continuous and differentiable latent state $\mathbf{z}(\tau)$. Within an infinitesimally small step limit ϵ in the latent space, the local continuous dynamics are modeled as $\mathbf{z}(\tau + \epsilon) = \mathbf{z}(\tau) + \epsilon \cdot \frac{d\mathbf{z}(\tau)}{d\tau}$. The derivative of the latent state, $\frac{d\mathbf{z}(\tau)}{d\tau}$, is represented by a neural network $f(\mathbf{z}(\tau), \tau; \phi)$ parameterized by learnable parameters ϕ . The latent state at any arbitrary time τ_s is obtained by solving the ODE from the

initial time τ_0 :

$$\mathbf{z}(\tau_s) = \mathbf{z}(\tau_0) + \int_{\tau_0}^{\tau_s} f(\mathbf{z}(\tau), \tau; \phi) d\tau = \text{ODESolve}(\mathbf{z}(t_0), f, t_0, t_s, \phi). \quad (5.1)$$

The derivative f modeled by the neural network is expressed as a uniformly Lipschitz continuous non-linear function in $\mathbf{z}$ and τ [40]. Therefore, the solution obtained through the solver for any given integration interval (τ_i, τ_j) is always unique in the integration of the continuous dynamics. The simplest method to solve ODE is the Euler method [41], which is a fixed-step-size first-order solver. Additionally, the Runge-Kutta [42] methods are preferred as a higher-order solvers as they offer enhanced stability.

The derivative f modeled by the neural network is expressed as a uniformly Lipschitz continuous non-linear function in $\mathbf{z}$ and τ [40]. Therefore, the solution obtained through the solver for any given integration interval (τ_i, τ_j) is always unique in the integration of the continuous dynamics. We model the 3D camera trajectory as time-continuous using neural ODEs in the latent space for a continuous representation. In our experiments, we adopt the fourth-order Runge-Kutta solver for all experiments, following [11].

5.3 Method

5.3.1 CoMoGaussian Framework

Our goal is to reconstruct a sharp 3D scene using only camera motion-blurred images as input, thereby obtaining deblurred novel view images. Inspired by image blind deblurring methods, we follow the approach of Deblur-NeRF [1], learning a kernel that intentionally blurs images and excluding this kernel during rendering to produce sharp images. Since sharp image supervision is not available during training, we adhere to the fundamental principle of accurately simulating motion-blurred images. In this framework, learning to reconstruct blurry images inherently drives the model to capture the underlying sharp image representation. As illustrated in Fig. 5.1, our blurring process consists of camera poses along the camera motion trajectory, generated continuously in time order through neural ODEs [11]. Each pose is composed of a rigid body transformation $\tilde{\mathbf{T}}_r = [\tilde{\mathbf{R}}_r | \tilde{\mathbf{t}}_r]$ (Sec. 5.3.2), which maintains the shape and size of the subject, and a continuous motion refinement (CMR) transformation $\tilde{\mathbf{T}}_c = [\tilde{\mathbf{R}}_c | \tilde{\mathbf{t}}_c]$ (Sec. 5.3.3), which compensates for the limitations of approximating continuous motion using a discrete sum of rigid transformations by introducing learnable adjustments. Note that $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ and $\mathbf{t} \in \mathbb{R}^3$ represent rotation matrix and translation vector, and $\tilde{\mathbf{R}} \in \mathbb{R}^{3 \times 3}$ and $\tilde{\mathbf{t}} \in \mathbb{R}^3$ stand for multiplicative offsets of change of the rotation matrix and translation vector, respectively. Given the two transformation matrices $\tilde{\mathbf{T}}_r = [\tilde{\mathbf{R}}_r | \tilde{\mathbf{t}}_r]$ and $\tilde{\mathbf{T}}_c = [\tilde{\mathbf{R}}_c | \tilde{\mathbf{t}}_c]$, the subsequent pose

$\mathbf{T}^{\tau_s} (0 \leq s < N)$ in the camera motion is derived as follows:

$$\mathbf{T}^{\tau_s} = \mathbf{T}^{\tau_M} \tilde{\mathbf{T}}_r^{\tau_s} \tilde{\mathbf{T}}_c^{\tau_s}, \text{ where } \mathbf{T} = [\mathbf{R}|\mathbf{t}] = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ 0 & 1 \end{bmatrix}, \quad (5.2)$$

where $\mathbf{T}^{\tau_M} (M = N/2)$ denotes given calibrated camera pose. We assume that calibrated camera pose is located at the midpoint of the camera motion trajectory, as it represents an averaged position influenced by the integration of motion blur over the exposure time. By rendering N images from the obtained N camera poses and computing their pixel-wise weighted sum (Sec. 5.3.4), we obtain the final blurred image. Our framework is shown in Fig. 5.2.

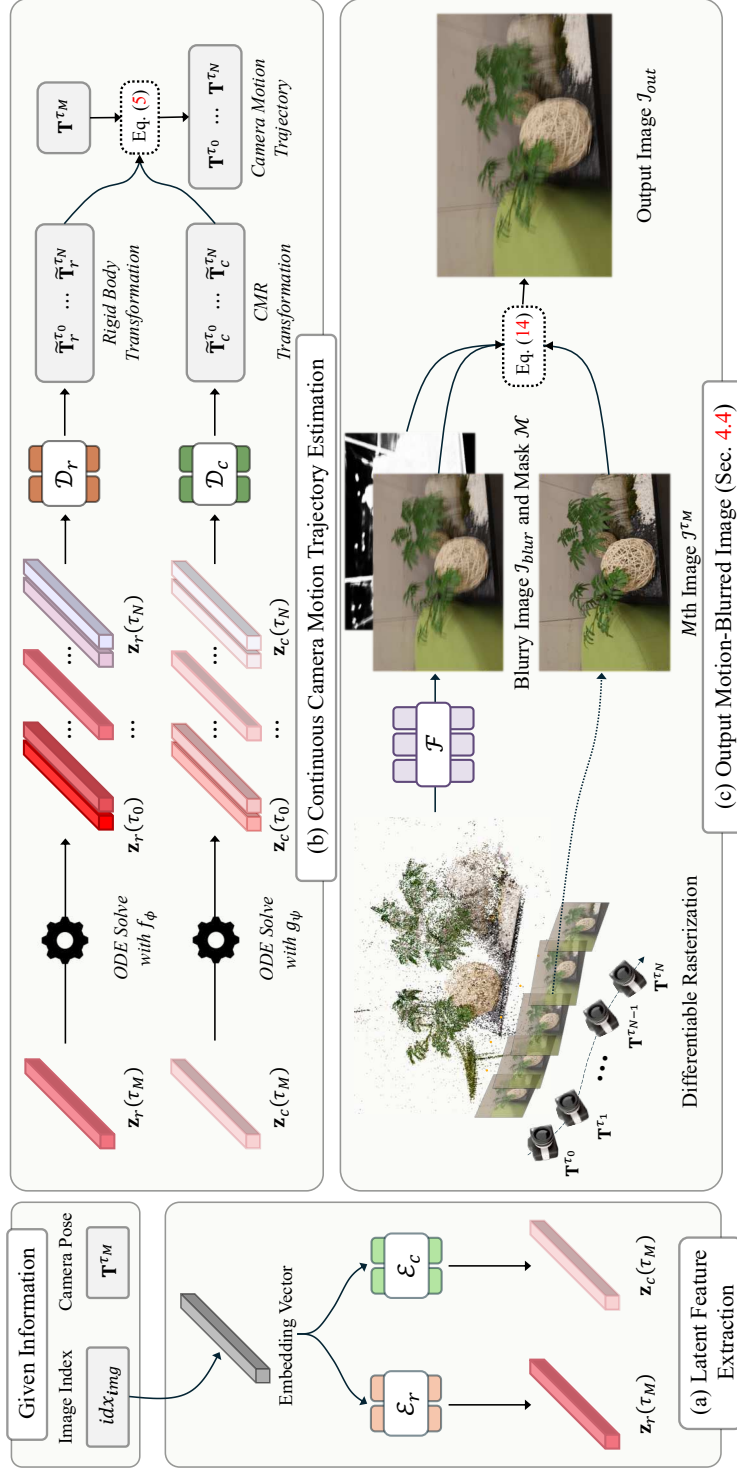


Figure 5.2: Pipeline of CoMoGaussian. (a) The given image index is embedded and passed through the encoders $\mathcal{E}_r$ and $\mathcal{E}_c$ to obtain latent features for the rigid body transformation and the CMR transformation. (b) CoMoGaussian solves the ODEs using the latent features and the neural derivatives f_ϕ and g_ψ , and obtains N transformed camera poses via Eq. (5.2). (c) Then, N images are rendered through Mip-Splatting [2], and the output motion-blurred image is obtained through the pixel-wise weighted-sum and scalar pixel mask (Sec. 5.3.4).

5.3.2 Continuous Rigid Body Motion

To design the camera motion trajectory within the exposure time, we apply rigid body transformation in that the shape and size of the object remain unchanged. Rigid body transformation requires three components for the unit screw axis $\mathcal{S} = (\hat{\omega}, v)$: the unit rotation axis $\hat{\omega} \in \mathbb{R}^3$, the rotation angle θ about the axis, and the translation component $v \in \mathbb{R}^3$ for translation. Considering that the direction and extent of blur vary for each image in a single scene, we embed the image index of the scene to obtain different embedded features for each image as shown in Fig. 5.2. These features are then passed through an encoder $\mathcal{E}_r$ with a single-layer MLP, transforming them into the latent features $\mathbf{z}_r(\tau_0)$, which potentially represent for $\hat{\omega}$, θ , and v of $\mathcal{S}$. Then, we model the continuous latent space for the screw axis using a neural ODEs [11] to assign the latent continuity to the screw axis in camera motion trajectory. The neural derivative f of the latent features for the screw axis is expressed as $\frac{d\mathbf{z}_r(\tau)}{d\tau} = f(\mathbf{z}_r(\tau), \tau; \phi)$, and the latent features at an arbitrary time τ_s can be obtained by ODE solver, numerically integrating f from τ_M to τ_s :

$$\mathbf{z}_r(\tau_s) = \mathbf{z}_r(\tau_M) + \int_{\tau_M}^{\tau_s} f(\mathbf{z}_r(\tau), \tau; \phi) d\tau. \quad (5.3)$$

To obtain N poses along the camera trajectory, we uniformly sample N time points within the fixed exposure time and get N latent features by applying the Eq. (5.3) to them. We then transform the latent features obtained by the ODE solver into the unit screw axis $\mathcal{S}$ using a single-layer MLP decoder $\mathcal{D}_r$. In Nerfies [43] and DP-

NeRF [10], the angular velocity $\omega = \hat{\omega}\theta$ is modeled and then decomposed into the unit rotation axis $\hat{\omega}$ and the rotation angle θ , making the two elements dependent on each other. However, since θ represents the rotation amount about the axis, these two elements should be independent. Therefore, we model $\hat{\omega}$ with normalization and θ independently through the decoder $\mathcal{D}_r$:

$$\mathcal{D}_r(\mathbf{z}_r(\tau)) = (\hat{\omega}^\tau, \theta^\tau, v^\tau), \text{ where } \|\hat{\omega}^\tau\| = 1. \quad (5.4)$$

According to [44], the screw axis $\mathcal{S}^\tau = (\hat{\omega}^\tau, v^\tau)$ is a normalized twist, so the infinitesimal transformation matrix $[\mathcal{S}^\tau] \in \mathbb{R}^{4 \times 4}$ is represented as follows:

$$[\mathcal{S}^\tau] = \begin{bmatrix} [\hat{\omega}^\tau] & v^\tau \\ 0 & 0 \end{bmatrix} \in \mathfrak{se}(3), \quad (5.5)$$

where $[\hat{\omega}^\tau] \in \mathfrak{so}(3)$ is a 3×3 skew-symmetric matrix of vector $\hat{\omega}^\tau$. To derive the infinitesimal transformation matrix $[\mathcal{S}^\tau]\theta^\tau \in \mathfrak{se}(3)$ on the Lie Algebra to the transformation matrix $\mathbf{T}_r^\tau \in SE(3)$ in the Lie Group, we use the matrix exponential $e^{[\mathcal{S}^\tau]\theta^\tau}$, whose rotation matrix and translation vector are $\mathbf{R}_r^\tau = e^{[\hat{\omega}^\tau]\theta^\tau}$ and $\mathbf{t}_r^\tau = G(\theta^\tau)v^\tau$, respectively. These matrices are expressed as follows using Rodrigues' formula [45] through Taylor expansion:

$$e^{[\hat{\omega}^\tau]\theta^\tau} = I + \sin \theta^\tau [\hat{\omega}^\tau] + (1 - \cos \theta^\tau) [\hat{\omega}^\tau]^2 \in SO(3), \quad (5.6)$$

$$G(\theta^\tau) = I\theta^\tau + (1 - \cos \theta^\tau)[\hat{\omega}^\tau] + (\theta^\tau - \sin \theta^\tau)[\hat{\omega}^\tau]^2. \quad (5.7)$$

Through the whole process, we obtain N continuous rigid body transformation matrices $\mathbf{T}_r = e^{[S]\theta}$, and by multiplying these with the input pose as in Eq. (5.2), we obtain the transformed poses.

5.3.3 Continuous Motion Refinement

We predict the continuous camera trajectory using only rigid body transformations; however, the computation of blurry pixels through numerical integration is inherently discrete. In other words, while a blurry pixel should ideally be formed by continuous integration over time, we approximate it using only N discrete samples. Consequently, rigid body transformations alone are insufficient to accurately represent the continuous motion of the camera under discretized numerical integration. To address this limitation, we introduce the CMR Transformation. This transformation has higher degrees of freedom from a learning perspective and is simple to implement. The CMR transformation optimizes the transformation matrix close to $SE(3)$ without explicitly enforcing its constraints. Unlike traditional rigid body transformations that strictly adhere to $SE(3)$ by parameterizing motion through a screw axis representation $(\hat{\omega}, v)$, the CMR transformation learns a transformation matrix $\tilde{\mathbf{R}}_c$ and a translation vector $\tilde{\mathbf{t}}_c$ in an unconstrained space. This process begins by encoding the image index into the latent state $\mathbf{z}_c(\tau_M)$ of $\tilde{\mathbf{R}}_c$ and $\tilde{\mathbf{t}}_c$ using the encoder $\mathcal{E}_c$ with a single-layer MLP, similar to Sec. 5.3.2. The latent state $\mathbf{z}_c(\tau_s)$ at any arbitrary time

τ_s is obtained using the neural derivative g parameterized by ψ and a solver:

$$\mathbf{z}_c(\tau_s) = \mathbf{z}_c(\tau_M) + \int_{\tau_M}^{\tau_s} g(\mathbf{z}_c(\tau), \tau; \psi) d\tau. \quad (5.8)$$

The latent features are then decoded into rotation matrix component $\mathbf{A}_c^\tau$ and translation vector $\tilde{\mathbf{t}}_c^\tau$ through a single-layer MLP decoder $\mathcal{D}_c$:

$$\mathcal{D}_c(\mathbf{z}_c(\tau)) = (\mathbf{A}_c^\tau, \tilde{\mathbf{t}}_c^\tau) \rightarrow \tilde{\mathbf{T}}_c^\tau = \begin{bmatrix} \tilde{\mathbf{R}}_c^\tau & \tilde{\mathbf{t}}_c^\tau \\ 0 & 1 \end{bmatrix}, \quad (5.9)$$

where $\tilde{\mathbf{R}}_c^\tau = \mathbf{A}_c^\tau + \mathbf{I}$ and $\mathbf{I}$ is the identity matrix to initialize $\tilde{\mathbf{R}}_c^\tau$ to identity transformation. Since the CMR transformation is intended to refine the rigid body transformation, it should not significantly affect the rigid body transformation during the initial stages of training. Therefore, we initialize the weights of the decoder to approximate $\mathbf{A}_c^\tau$ to zero matrix using a uniform distribution $\mathcal{U}(-10^{-5}, 10^{-5})$, which means that $\tilde{\mathbf{R}}_c^\tau$ is initialized close to identity matrix. To ensure that $\tilde{\mathbf{R}}_c$ remains close to a valid rotation matrix, we apply explicit regularization, allowing for greater flexibility while maintaining consistency with rigid motion. The proposed loss is $\mathcal{L}_o$, which ensures the orthogonality condition of rotation matrix: $\mathcal{L}_o = \|\tilde{\mathbf{R}}_c^\top \tilde{\mathbf{R}}_c - \mathbf{I}\|_2$. Finally, we get the refined transformed camera pose $\mathbf{T}^{\tau_s}$ using $\tilde{\mathbf{T}}_c^{\tau_s}$ and $\tilde{\mathbf{T}}_r^{\tau_s}$ obtained from Sec. 5.3.2.

5.3.4 Optimization

Pixel-wise Weight and Mask. Once the N images along the camera motion trajectory are rendered from the N camera poses, we apply a pixel-wise weighted sum to create the blurry image $\mathcal{I}_{blur}$, following previous research [1,10,22,24]. To satisfy Eq. (3.7), we use a shallow CNN $\mathcal{F}$ and a softmax function to compute the pixel-wise weights $\mathcal{P} \in \mathbb{R}^{N \times H \times W \times 3}$ for the resulting images, as follows:

$$\mathcal{I}_{blur} = \sum_{i=1}^N \mathcal{I}^{\tau_i} \cdot \mathcal{P}^{\tau_i}, \text{ where } \mathcal{P} = \text{softmax}(\mathcal{F}(\mathcal{I})), \quad (5.10)$$

where $\mathcal{I}^{\tau_i}$ is the i -th image along the camera motion, and $\mathcal{P}^{\tau_i}$ is the pixel-wise weight of i -th image. Additionally, we adopt per-pixel scalar mask [9] $\mathcal{M} \in \mathbb{R}^{H \times W \times 3}$ to generate final output blurry image $\mathcal{I}_{out}$ by blending the sharp image $\mathcal{I}^{\tau_M}$ at camera pose $\mathbf{T}^{\tau_M}$ and the blurry image $\mathcal{I}_{blur}$ acquired by pixel-wise weight:

$$\mathcal{I}_{out} = (\mathbf{1} - \mathcal{M}) \cdot \mathcal{I}^{\tau_M} + \mathcal{M} \cdot \mathcal{I}_{blur}. \quad (5.11)$$

The scalar mask aims to decide whether each pixel is blurry or not, and a small sparsity constraint is applied to the mask to assign a larger weight on $\mathcal{I}^{\tau_M}$, namely mask sparsity loss $\mathcal{L}_{\mathcal{M}}$, which is a mean value of the scalar mask. This method allows us to obtain the final precise blurry image, which is then optimized against the ground truth blurry image.

Objective. We optimize the learning process using the $\mathcal{L}_1$ loss and D-SSIM between the generated output blurry image $\mathcal{I}_{out}$ and the ground truth blurry image, similar to 3D-GS [7]. The $\mathcal{L}_1$ loss ensures pixel-wise accuracy, while D-SSIM captures perceptual differences. Additionally, we apply the regularization loss $\mathcal{L}_o$ to regularize the CMR transformation and mask sparsity loss $\mathcal{L}_M$. The final objective $\mathcal{L}$ is defined as follows:

$$\mathcal{L} = (1 - \lambda_c)\mathcal{L}_1 + \lambda_c\mathcal{L}_{D-SSIM} + \lambda_o\mathcal{L}_o + \lambda_M\mathcal{L}_M, \quad (5.12)$$

where λ_c is a factor for balancing $\mathcal{L}_1$ and $\mathcal{L}_{D-SSIM}$, and λ_o and λ_M are factors for $\mathcal{L}_o$ and $\mathcal{L}_M$, respectively.

5.4 Experiments

Datasets. CoMoGaussian is evaluated using three datasets: the Deblur-NeRF [1] synthetic dataset, the Deblur-NeRF real-world dataset, and the ExbluRF [3] real-world dataset. The Deblur-NeRF synthetic dataset consists of 5 scenes generated using Blender [31], where the images are combined in linear RGB space to create the final blurred images. The Deblur-NeRF real-world dataset includes 10 scenes captured with a CANON EOS RP camera, where the exposure time is manually set to produce blurry images. The ExbluRF real-world dataset consists of 8 scenes with challenging camera motion. We obtain the camera poses for each image and the initial point clouds by applying COLMAP [32,33].

Implementation Details CoMoGaussian is trained for 40k iterations based on Mip-Splatting [2]. We set the number of poses N that constitute the continuous camera trajectory to 9, which indicates that M is 5. The embedding function of Sec. 5.3.2 is implemented by `nn.Embedding` of PyTorch, and the embedded features have the sizes of hidden state of 64. The sizes of hidden state of the single-layer encoders $\mathcal{E}_r$, $\mathcal{E}_c$, and the single-layer decoders $\mathcal{D}_r$, $\mathcal{D}_c$ are also 64. $\mathcal{D}_r$ consists of three head MLPs that extract the screw axis parameters ω and v , along with θ , while $\mathcal{D}_c$ comprises two head MLPs that extract $\mathbf{A}_c$ and $\mathbf{t}_c$ as described in Sec. 5.3.3. The neural derivative function f consists of two parallel single MLP layers, where one is designated for the rotation component and the other for the translation component, where f and g share the learnable parameters. The CNN $\mathcal{F}$ consists of two convolutional layers with 32 channels with kernel size of 5×5 for the first layer and 3×3 for the rest one. The pixel-wise weights are obtained by applying a pointwise convolutional layer to the output of $\mathcal{F}$, and the scalar mask $\mathcal{M}$ is obtained by averaging it to the batch axis and applying another pointwise convolution to the output of $\mathcal{F}$. For first 1k iterations, Gaussian primitives are roughly trained without rigid body transformation and CMR transformation. After 1k iterations, those transformations start to be trained without the pixel-wise weight and the scalar mask to allow the initial camera motion path to be sufficiently optimized. After 3k iterations, the pixel-wise weight and the scalar mask start training. We set λ_c , λ_o , and $\lambda_{\mathcal{M}}$ to 0.3, 10^{-4} , and 10^{-3} respectively for the objective function. All experiments are conducted on a single NVIDIA RTX 4090 GPU.

5.4.1 Novel View Synthesis Results

For quantitative results, we evaluate CoMoGaussian using three metrics: peak signal-to-noise ratio (PSNR), structural similarity index measure (SSIM), and learned perceptual image patch similarity (LPIPS). We compare our method with both ray-based [1,4,10] and rasterization methods [2,8,9,25]. The overall results for all datasets are shown in Tabs. 5.1 and 5.2, demonstrating that our method achieves superior performance compared to all other methods. Additionally, benefiting from using Mip-Splatting [2] as our backbone, our proposed method ensures fast training and rendering speeds. We also observe that directly applying an image deblurring network, Restormer [34], followed by Mip-Splatting improves the performance over naive 3DGS, but still falls far behind dedicated 3D deblurring methods. This indicates that independently restoring each input image cannot fully recover view-consistent geometry and appearance, as image-space deblurring does not explicitly model the 3D blur formation process or camera motion during exposure. Although BAD-Gaussians [25] achieves relatively good LPIPS scores on ExbluRF dataset, their PSNR and SSIM scores are lower, indicating that their camera poses are not properly optimized.

To qualitatively evaluate the results, we visualize rendering results of several scenes and compare to other methods as shown in Fig. 5.3. Our method shows superior qualitative results compared to the state-of-the-art rasterization-based methods such as BAD-Gaussians [25], BAGS [9], and Deblurring 3DGS [8]. The bottom two rows of Fig. 5.3 stand for visualizations of ExbluRF [3] STONE LANTERN and SUN-FLOWERS scenes, where our CoMoGaussian best preserves the edge details of the

Table 5.1: Comparisons on Deblur-NeRF synthetic and real-world scene dataset. “*” denotes the results obtained by reproducing the released code. The orange and yellow cells respectively indicate the highest and second-highest value.

Methods	Synthetic Scene [1]			Real-World Scene [1]		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Naive NeRF [4]	23.78	0.6807	0.3362	22.69	0.6347	0.3687
Mip-Splatting [2]	23.28	0.6765	0.2822	21.87	0.6270	0.3066
Restormer [34] + Mip-Splatting	-	-	-	22.45	0.6845	0.2414
Deblur-NeRF [1]	28.77	0.8593	0.1400	25.63	0.7645	0.1820
DP-NeRF [10]	29.23	0.8674	0.1184	25.91	0.7751	0.1602
BAD-Gaussians* [25]	22.01	0.6377	0.1001	21.69	0.6471	0.1262
Deblurring 3DGS [8]	28.24	0.8580	0.1051	26.61	0.8224	0.1096
BAGS [9]	27.34	0.8353	0.1116	26.70	0.8237	0.0956
CoMoGaussian	31.02	0.9167	0.0492	27.85	0.8431	0.0822

Table 5.2: Comparisons on ExbluRF real-world scene dataset.

Methods	ExbluRF Real-World Scene [6]		
	PSNR↑	SSIM↑	LPIPS↓
Mip-Splatting [2]	24.52	0.5662	0.6003
ExbluRF [3]	23.45	0.5495	0.3805
BAD-Gaussians [25]	26.83	0.6625	0.3221
Deblurring 3DGS [8]	27.36	0.6795	0.3989
BAGS [9]	24.70	0.5843	0.5278
CoMoGaussian	30.15	0.7559	0.3107

ground truth, even when given input images with extreme motion blur.

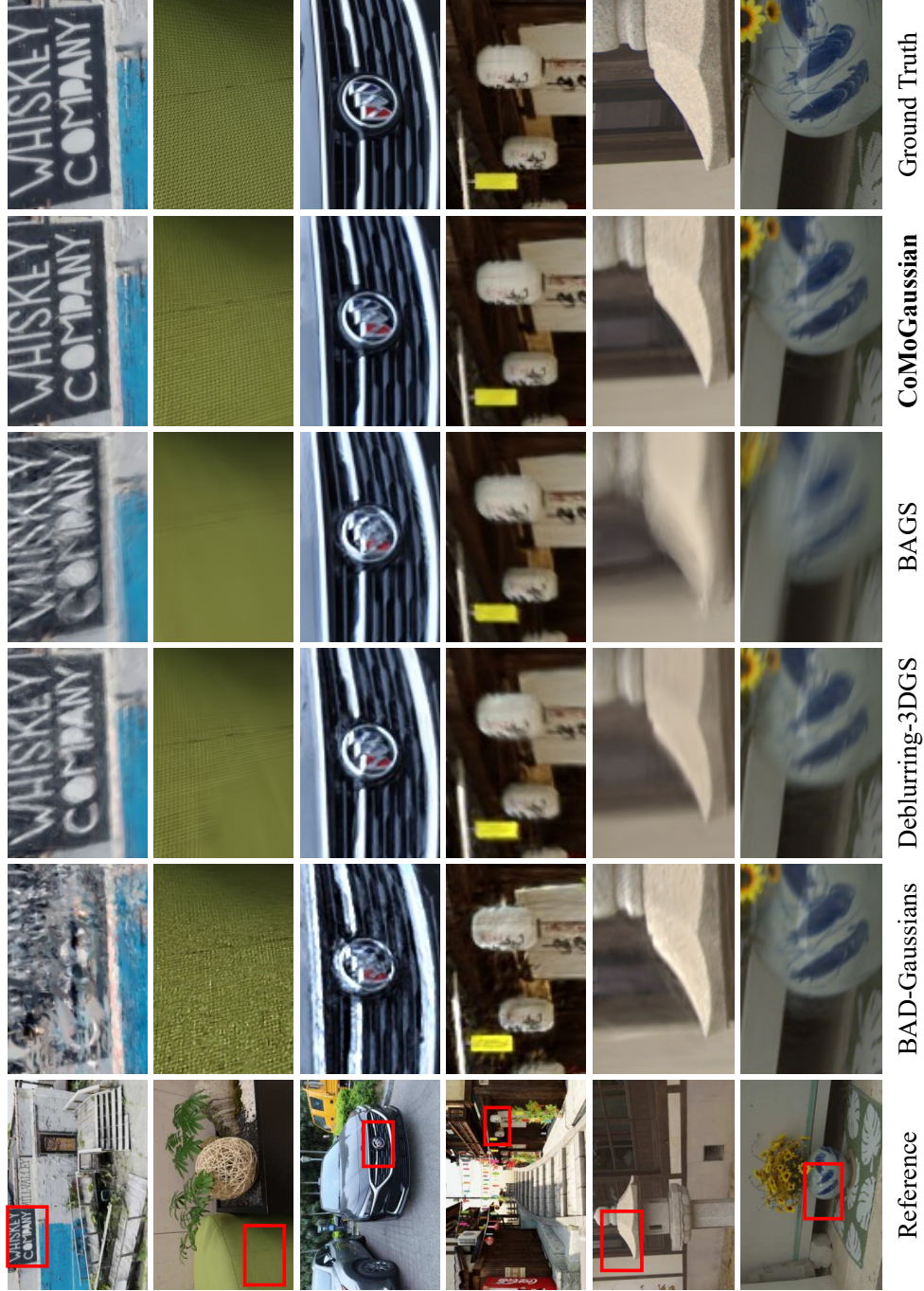


Figure 5.3: Qualitative comparison on the Deblur-NeRF [1] synthetic and real-world scenes, and ExbluRF [3] real-world scenes.

5.4.2 Ablation Study

To thoroughly demonstrate the effectiveness of rigid body and CMR transformations, we conduct ablation experiments as shown in Tab. 5.3. Additionally, we perform further ablation studies for our continuous modeling, presented in Tab. 5.4. All the experiments for Tabs. 5.3 and 5.4 are conducted on Deblur-NeRF [1] real-world scenes.

Table 5.3: Ablation study on our main components.

Rigid Motion	CMR	Orth. Loss	PSNR↑	SSIM↑	LPIPS↓
			21.87	0.6270	0.3066
✓			27.19	0.8249	0.1005
✓	✓		27.74	0.8402	0.0878
✓	✓	✓	27.85	0.8431	0.0822

Ablation on Transformations. We analyze the impact of the main components of CoMoGaussian: the rigid body transformation and the CMR transformation. The baseline in Tab. 5.3 is Mip-Splatting [2], and when pixel-wise weights are not applied, the N rendered images are averaged to produce a blurry image. We observe that incorporating the rigid body transformation alone into the baseline results in significantly improved performance across all metrics compared to the baseline. Furthermore, adding the CMR transformation to the rigid body motion, which compensates for the discretized integration, consistently enhances performance across all metrics. Finally, applying the orthogonality loss $\mathcal{L}_o$, a component of the CMR transformation, leads to further improvements in LPIPS.

Table 5.4: Ablation study on the type of camera motion estimator.

Methods	Physical Space	Latent Space	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Linear Interpolation [25]	✓		20.97	0.6085	0.1461
Cubic B-Spline [25]	✓		21.69	0.6471	0.1262
MLP		✓	27.43	0.8295	0.1010
GRU		✓	27.40	0.8302	0.0988
Neural ODE		✓	27.85	0.8435	0.0822

Ablation on Neural ODE. To validate the effectiveness of implementing rigid body and CMR transformations continuously in latent space, we conduct ablation studies using various approaches, as shown in Tab. 5.4. For linear interpolation and cubic B-spline, we adopt the implementations from BAD-Gaussians [25]. Since these methods operate in 3D physical space rather than latent space, they exhibit lower performance compared to other latent space-based methods. For latent space-based methods, we test MLP and GRU [46] as alternatives to neural ODEs. MLP-based method replaces the neural ODE by extracting N latent features from the latent feature of τ_M using a two-layer MLP. GRU-based method replaces the neural ODE with GRU cells to generate latent features. Our neural ODE-based approach consistently outperforms both methods. Notably, in our neural ODE framework, we extract features for all time points by propagating forward ($\tau_M \rightarrow \tau_N$) and backward ($\tau_M \rightarrow \tau_0$) from the midpoint τ_M of the exposure time. Since all these processes share the same neural derivative, they remain within the same function space. However, in the GRU-based approach, forward and backward time features require separate GRU cells, making it theoretically less suitable than our neural ODE-based method.

Table 5.5: Experimental results on NeRF-LLFF [4,5] dataset.

Methods	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Mip-Splatting [2]	27.71	0.8685	0.0494
Deblurring 3DGS [8]	17.42	0.5075	0.3187
BAGS [9]	26.35	0.8490	0.0528
CoMoGaussian	27.56	0.8669	0.0457

Experiments on Sharp Images. To evaluate the generalization capability of CoMoGaussian, we conduct experiments on the NeRF-LLFF [4,5] dataset, a forward-facing scenes consisting of sharp images, with the results presented in Tab. 5.5. CoMoGaussian performs comparably to Mip-Splatting [2], demonstrating that our method is not limited to motion-blurred images. Additionally, we evaluate the performance of other methods, Deblurring 3DGS [8] and BAGS [9]. They struggle to optimize certain scenes and fall short of CoMoGaussian in quantitative performance. This highlights the superior generalization ability of our approach compared to existing methods.

Ablation on ODE solvers. We further analyze the effect of different ODE solvers for modeling the continuous camera trajectory during exposure. As shown in Tab. 5.6, we compare fixed-step solvers, including Euler, second-order Runge–Kutta, and fourth-order Runge–Kutta, with adaptive-step solvers, including fifth-order and eighth-order Dopri–Prince methods. Overall, fixed-step solvers achieve competitive performance while maintaining the same training time. Among them, the second-order Runge–

Table 5.6: Ablation study on ODE solvers for continuous camera trajectory modeling (Deblur-NeRF ball scene).

Solver (Method)		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Training Time
Fixed Step	Euler	29.51	0.8446	0.1182	2h 23m
	2nd Runge-Kutta	29.63	0.8460	0.1176	2h 23m
	4th Runge-Kutta	29.60	0.8422	0.1115	2h 23m
Adaptive Step	5th Dopri-Prince	29.49	0.8458	0.1156	3h 40m
	8th Dopri-Prince	29.55	0.8435	0.1159	8h 10m

Kutta solver obtains slightly higher PSNR and SSIM, whereas the fourth-order Runge-Kutta solver achieves the best LPIPS score. Since LPIPS better reflects perceptual reconstruction quality, we adopt the fixed-step fourth-order Runge-Kutta solver as our default ODE solver. In contrast, adaptive-step solvers do not provide consistent performance gains despite requiring substantially longer training time. This suggests that the camera trajectory within a short exposure interval can be sufficiently approximated by a fixed-step solver, while the additional computational overhead of adaptive solvers is not beneficial in our setting.

5.4.3 Camera Trajectory Visualization

We visualize the continuous camera trajectories for the HERON (left) and STAIR (right) scenes from the Deblur-NeRF real-world dataset, as shown in Fig. 5.4. The camera trajectory for a single motion-blurred image is represented by colored cones, with the cone’s color gradually transitioning from red to light purple as time progresses from t_0 to t_N . The visualized trajectories confirm that the camera paths generated by CoMoGaussian are smoothly continuous, validating that the continuous

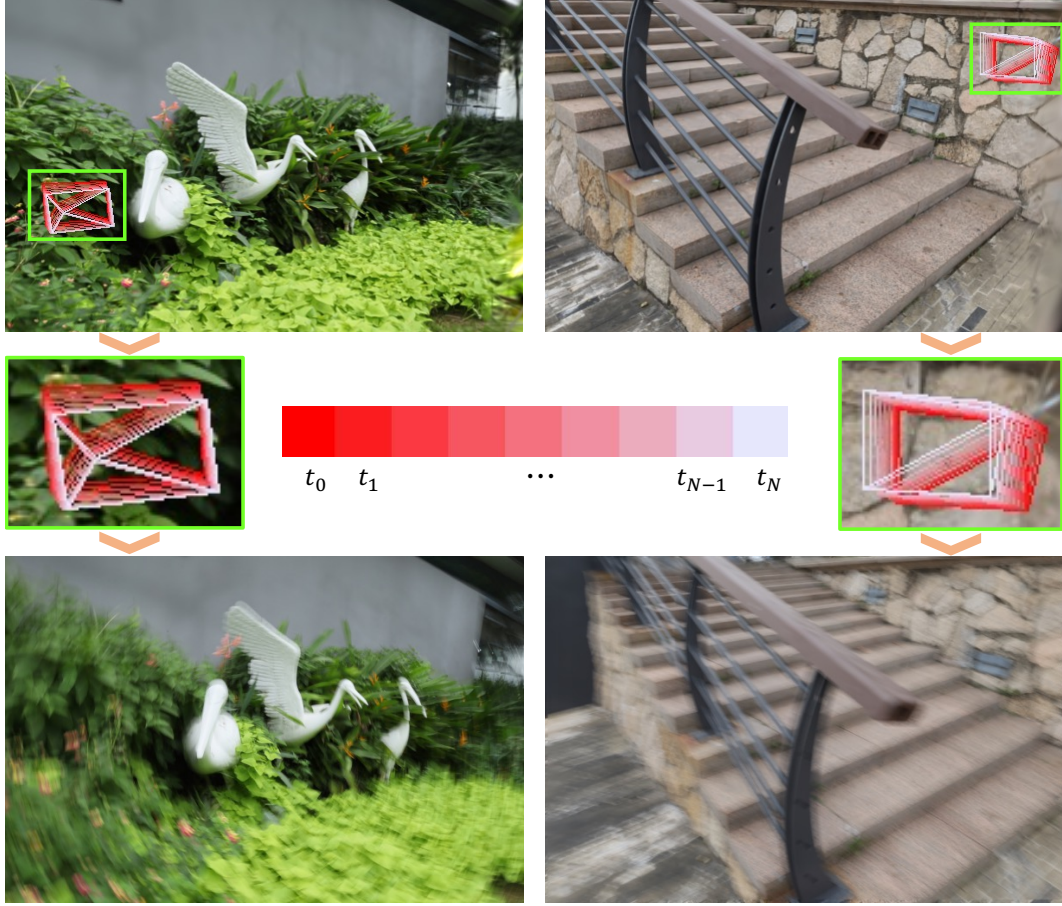


Figure 5.4: Camera Trajectory Visualization. Red cones stand for camera poses, and the images below are the output blurry images.

transformations described in Secs. 5.3.2 and 5.3.3 function as intended. Additionally, the output blurry images in Fig. 5.4 align with the visualized camera poses, accurately reflecting both the positions and directions of the camera. Specifically, even though the camera path in the HERON scene on the left is nonlinear, CoMoGaussian accurately predicts this path and generates a precise blurry image.

Visualization for Other Estimators. To qualitatively evaluate Tab. 5.4, we visualize the camera poses generated by the MLP-based and GRU-based estimators in Fig. 5.5. Since the MLP estimator does not account for the sequential nature of camera motion, it inherently lacks continuity, resulting in a trajectory that appears discontinuous and inconsistent. While the GRU estimator produces trajectories that seem more continuous, it separately implements GRU cells for forward and backward propagation, causing discontinuities between camera poses along the trajectory. In contrast, our neural ODE-based estimator ensures a fully continuous camera motion over time, achieving both visually smooth trajectories and superior quantitative performance compared to the MLP and GRU estimators.

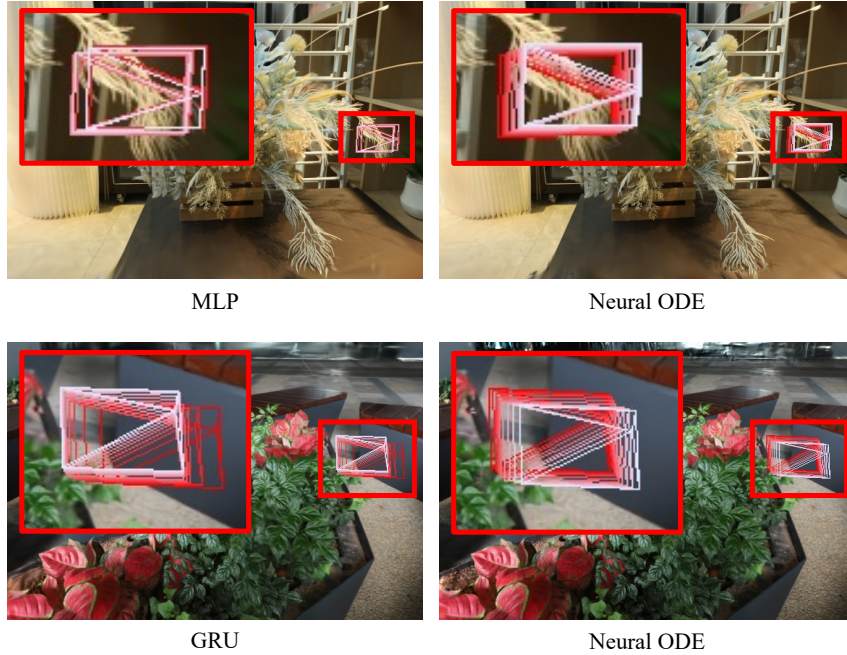


Figure 5.5: Comparison between the Neural ODE-based estimator and other estimators. Only the Neural ODE-based estimator exhibits a continuous camera trajectory.

Visualization for Sharp Images. We visualize the camera poses to examine how CoMoGaussian’s camera trajectory modeling operates on the NeRF-LLFF dataset [4,5], which consists of sharp images. As shown in Fig. 5.6, the camera motion trajectory for sharp images remains nearly stationary, demonstrating the generalization capability of our proposed method.

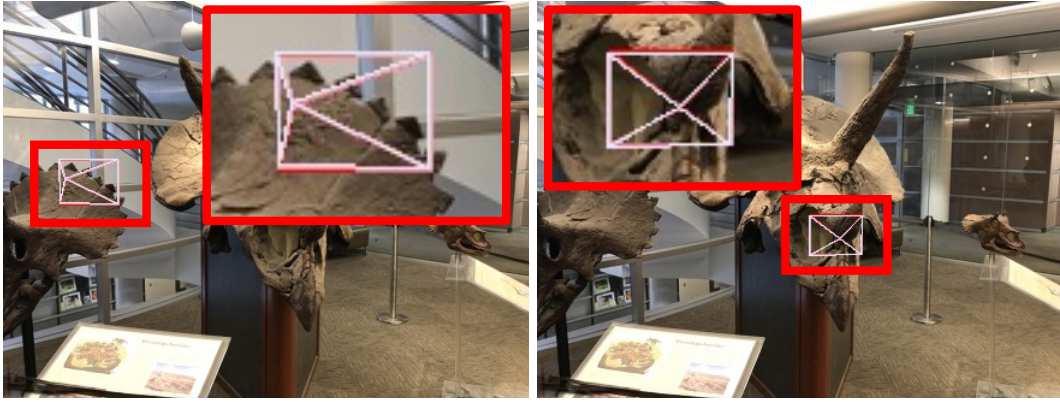


Figure 5.6: Visualization of camera motion trajectories for sharp images.

Additional Visualization We visualize the camera motion trajectories for input blurry images predicted by CoMoGaussian in Fig. 5.7 and Fig. 5.8. Fig. 5.7 illustrates camera motions for images with significant blur, where the predicted trajectories are continuous over time and align precisely with the input images. Fig. 5.8 depicts camera motions for images with relatively less blur, where the predicted trajectories show minimal movement, yet still match the input images accurately. These results demonstrate that our blurring kernel effectively models precise continuous camera motion.



Figure 5.7: Camera motion trajectory predicted by CoMoGaussian for input images with significant blur.

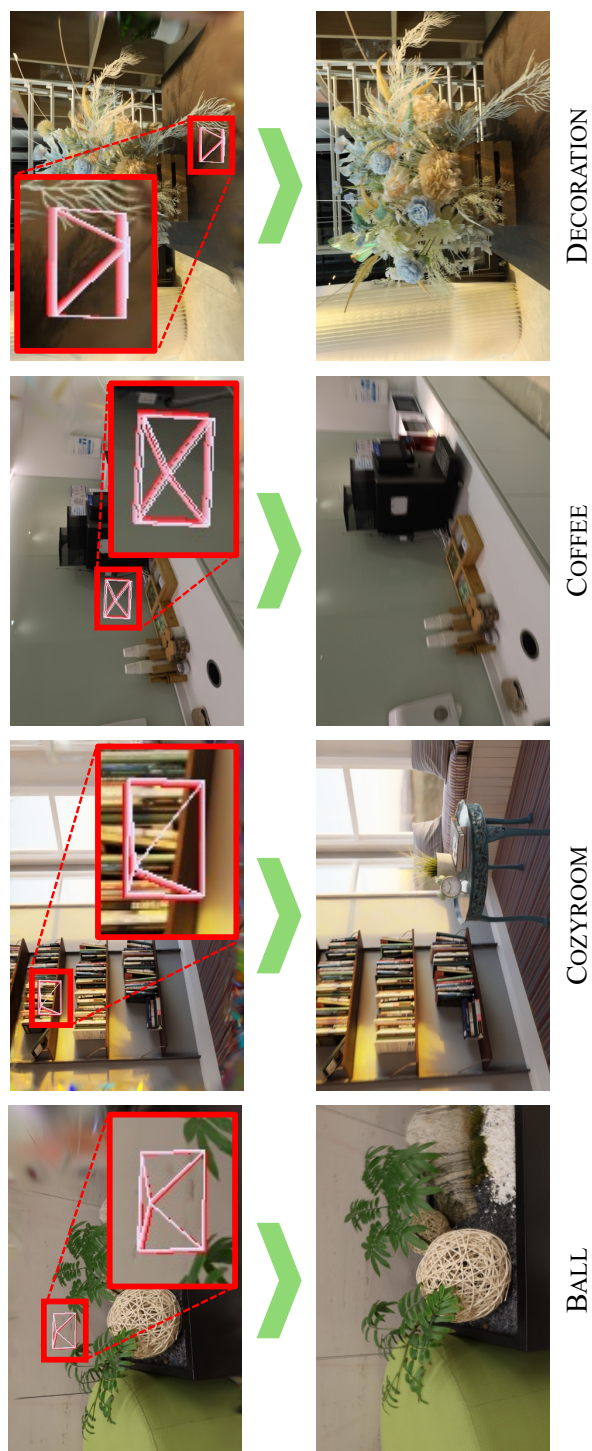


Figure 5.8: Camera motion trajectory predicted by CoMoGaussian for input images with moderate blur.

Table 5.7: Experimental results based on the number of poses N .

# Cam.	Rigid Only (Sec. 5.3.2)			Rigid (Sec. 5.3.2) + CMR (Sec. 5.3.3)		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
5	26.60	0.8101	0.1217	26.90	0.8245	0.1073
6	26.94	0.8187	0.1101	27.40	0.8345	0.0996
7	27.08	0.8218	0.1059	27.52	0.8377	0.0925
8	27.08	0.8241	0.1025	27.67	0.8399	0.0865
9	27.19	0.8249	0.1005	27.85	0.8435	0.0822
10	27.29	0.8267	0.0972	27.89	0.8442	0.0814
11	27.36	0.8295	0.0933	27.79	0.8423	0.0833
12	27.43	0.8312	0.0906	27.86	0.8399	0.0826
13	27.57	0.8341	0.0879	27.81	0.8406	0.0813

5.4.4 Number of Poses on Camera Motion.

We conduct an ablation study on the number of camera poses used to model continuous camera motion, with the results presented in Tab. 5.7. We perform two separate experiments: one using only the rigid body transformation described in Sec. 5.3.2, and the other incorporating both the rigid body transformation and the CMR transformation discussed in Sec. 5.3.3. The trends in the three evaluation metrics with respect to different values of N in Tab. 5.7 are visualized in Fig. 5.9.

When using only the rigid body transformation, performance improves almost linearly as N increases across all metrics. Since a motion-blurred image is fundamentally the result of integrating sharp images over time, increasing N allows for a finer discretization of the continuous camera trajectory, leading to better performance. However, numerical integration is inherently discrete, even when the camera motion itself is continuous, which explains why a larger N results in improved accuracy.

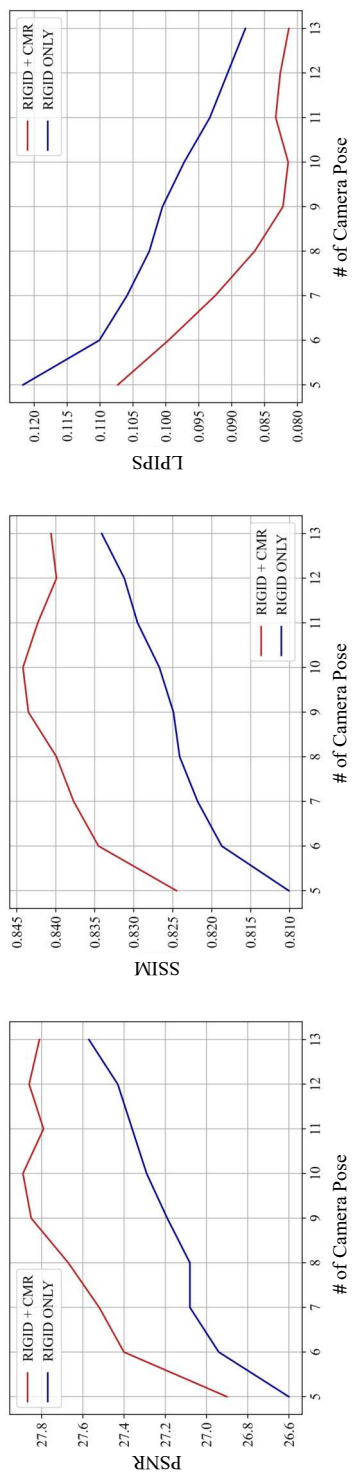


Figure 5.9: Performance variation based on the number of poses N used to construct the camera motion trajectory.

Nevertheless, increasing N also leads to a significant rise in computational complexity during training, as each motion-blurred image requires rendering N sharp images. This creates a trade-off: smaller values of N suffer from discretization artifacts, while larger values of N lead to increased computational costs due to multiple renderings. To address this, we introduce the CMR transformation, which adds a small degree of flexibility to the transformation matrix, effectively compensating for the limitations of discretized integration at smaller N .

As shown in Tab. 5.7 and Fig. 5.9, when CMR is applied alongside the rigid body transformation, performance saturates at $N = 9$ across all metrics, even surpassing the performance of the rigid body transformation alone at $N = 13$. This demonstrates that the CMR transformation mitigates the limitations of rigid body transformations at lower N and justifies its inclusion in our framework.

5.4.5 Orthogonal Regularization.

We conduct a qualitative ablation study on the regularization loss for the orthogonality condition introduced in Sec. 5.3.3 of the main paper. Although the impact of this loss may appear minor in Tab. 5.3 of the main paper, a qualitative comparison reveals a noticeable difference.

As shown in Fig. 5.10, including this regularization results in significantly improved visual quality compared to when it is omitted. Without regularization, the 3×3 linear transformation matrix is learned without constraints on shearing and scaling, which leads to unintended distortions. When such unconstrained affine trans-

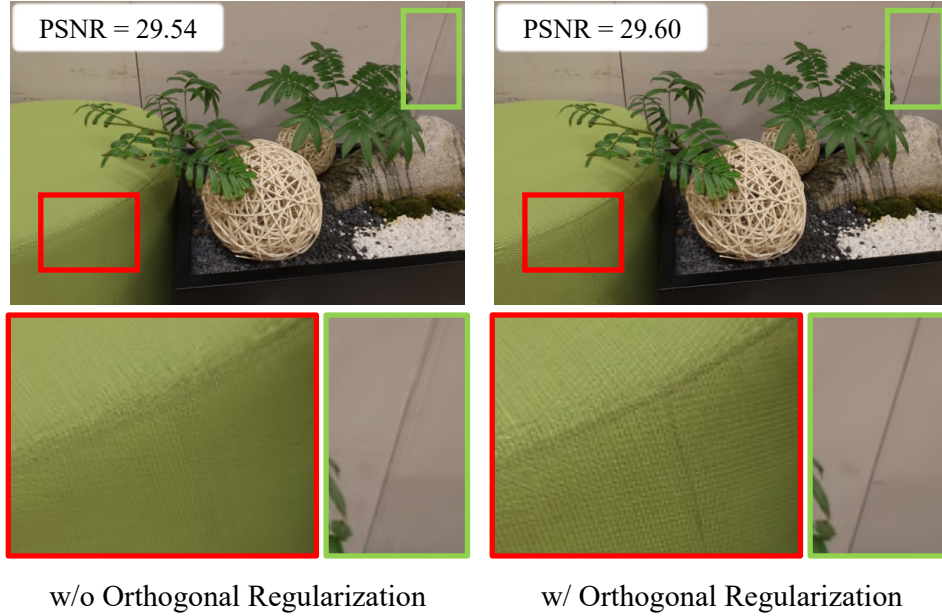


Figure 5.10: Qualitative comparison of results with and without orthogonal regularization. Despite similar PSNR values, the results with regularization capture more fine details.

formations are included, the overall scene structure may still be well captured, but finer details tend to be lost.

By enforcing the orthogonality condition, we restrict the transformation matrix to allow only minimal deviations, preventing excessive distortions. As demonstrated in Fig. 5.10, while the quantitative performance remains similar, the model captures finer details more effectively, highlighting the importance of this regularization.

5.4.6 Training and Rendering Speed

In this section, we compare the training time and rendering speed of our method with recent 3DGS-based approaches. As shown in Tab. 5.8, CoMoGaussian requires

Table 5.8: Comparison of training and rendering speeds across various 3DGS-based methods. * indicates that the speed is identical to that of the corresponding model.

Methods	Training Time (hours)	Rendering Speed
BAD-Gaussians [25]	0.37	*3DGS [7]
Deblurring 3DGS [8]	0.20	*3DGS [7]
BAGS [9]	0.83	*Mip-Splatting [2]
CoMoGaussian	1.33	*Mip-Splatting [2]

a longer training time compared to other methods. However, given the quantitative results in Tabs. 5.9 to 5.11, as well as the qualitative comparisons in our *supplementary videos*, our contributions remain significant. We believe that addressing the limitations discussed in the main paper will enable faster training in the future.

Additionally, since sharp rendering is performed solely using 3DGS [7] or Mip-Splatting [2] without additional modules, our method achieves fast rendering speeds comparable to those of other approaches.

5.4.7 Per-Scene Quantitative Results

We show the per-scene quantitative performance on Deblur-NeRF real-world, synthetic, and ExbluRF real-world dataset in Tab. 5.9, Tab. 5.10, and Tab. 5.11. CoMoGaussian demonstrates superior quantitative performance on most scenes in the real-world dataset.

Table 5.9: Per-Scene Quantitative Performance on Deblur-NeRF Real-World Scenes.

Real-World Scene	BALL		BASKET		BUICK		COFFEE		DECORATION			
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Naive NeRF [4]	24.08	0.6237	0.3992	23.72	0.7086	0.3223	21.59	0.6325	0.3502	26.48	0.8064	0.2896
Mip-Splatting [2]	23.22	0.6190	0.3400	23.24	0.6880	0.2880	21.46	0.6590	0.2660	24.73	0.7490	0.2880
Deblur-NeRF [1]	27.36	0.7656	0.2230	27.67	0.8449	0.1481	24.77	0.7700	0.1752	30.93	0.8981	0.1244
DP-NeRF [10]	27.20	0.7652	0.2088	27.74	0.8455	0.1294	25.70	0.7922	0.1405	31.19	0.9049	0.1002
BAD-Gaussians [25]	22.28	0.6032	0.2054	22.02	0.7004	0.1197	19.95	0.6127	0.1103	25.58	0.7965	0.0932
Deblurring 3DGS [8]	28.27	0.8233	0.1413	28.42	0.8713	0.1155	25.95	0.8367	0.0954	32.84	0.9312	0.0676
BAGS [9]	27.68	0.7990	0.1500	29.54	0.9000	0.0680	26.18	0.8440	0.0880	31.59	0.9080	0.0960
CoMoGaussian	29.60	0.8422	0.1115	30.78	0.9041	0.0761	27.23	0.8502	0.0742	33.04	0.9247	0.0578
Real-World Scene	GIRL		HERON		PARTERRE		PUPPET		STAIR			
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Naive NeRF [4]	20.07	0.7075	0.3196	20.50	0.5217	0.4129	23.14	0.6201	0.4046	22.09	0.6093	0.3389
Mip-Splatting [2]	19.87	0.7140	0.2780	19.43	0.5050	0.3320	22.28	0.5900	0.3210	22.05	0.6310	0.2670
Deblur-NeRF [1]	22.27	0.7976	0.1687	22.63	0.6874	0.2099	25.82	0.7597	0.2161	25.24	0.7510	0.1577
DP-NeRF [10]	23.33	0.8139	0.1498	22.88	0.6930	0.1914	25.86	0.7665	0.1900	25.25	0.7536	0.1505
BAD-Gaussians [25]	19.16	0.7037	0.1178	19.47	0.5264	0.1747	21.71	0.6154	0.1165	21.74	0.6506	0.1166
Deblurring 3DGS [8]	23.26	0.8390	0.1011	23.14	0.7438	0.1543	26.17	0.8144	0.1206	25.67	0.8051	0.0941
BAGS [9]	25.45	0.8690	0.0790	22.04	0.7150	0.1260	25.92	0.8190	0.0920	25.81	0.8040	0.0940
CoMoGaussian	27.08	0.8884	0.0733	23.18	0.7350	0.1326	26.11	0.8131	0.0872	27.09	0.8337	0.0658
										27.92	0.7792	0.0541

Table 5.10: Per-Scene Quantitative Performance on Deblur-NeRF Synthetic Scenes.

Synthetic Scene	FACTORY			CozyRoom			POOL			TANABATA			TROLLY		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
Naive NeRF [4]	19.32	0.4563	0.5304	25.66	0.7941	0.2288	30.45	0.8354	0.1932	22.22	0.6807	0.3653	21.25	0.6370	0.3633
Mip-Splatting [2]	18.21	0.4234	0.4769	25.25	0.7968	0.1646	30.57	0.8483	0.1456	21.54	0.6754	0.3075	20.82	0.6388	0.3165
Deblur-NeRF [1]	25.60	0.7750	0.2687	32.08	0.9261	0.0477	31.61	0.8682	0.1246	27.11	0.8640	0.1228	27.45	0.8632	0.1363
DP-NeRF [10]	25.91	0.7787	0.2494	32.65	0.9317	0.0355	31.96	0.8768	0.0908	27.61	0.8748	0.1033	28.03	0.8752	0.1129
BAD-Gaussians [25]	17.86	0.3892	0.1440	23.50	0.7396	0.0616	26.90	0.7296	0.1127	20.54	0.6379	0.0860	21.26	0.6921	0.0963
Deblurring 3DGS [8]	24.01	0.7333	0.2326	31.45	0.9222	0.0367	31.87	0.8829	0.0751	27.01	0.8807	0.0785	26.88	0.8710	0.1028
BAGS [9]	22.35	0.6639	0.2277	32.21	0.9359	0.0245	28.72	0.8404	0.0804	26.79	0.8735	0.1099	26.61	0.8627	0.1156
CoMoGaussian	29.32	0.8971	0.0563	33.34	0.9427	0.0239	32.45	0.8924	0.0705	29.53	0.9273	0.0408	30.45	0.9240	0.0546

Table 5.11: Per-Scene Quantitative Performance on the ExbluRF Real-World Scenes.

ExbluRF	BENCH				CAMELLIA				DRAGON				JARS			
	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓
Mip-Splatting [2]	24.58	0.5671	0.6190	0.6190	23.28	0.5151	0.5886	0.5886	28.65	0.5403	0.7002	0.7002	24.08	0.5335	0.6094	0.6094
ExbluRF [3]	24.75	0.5783	0.3003	0.3003	23.14	0.4925	0.3630	0.3630	24.26	0.4042	0.5641	0.5641	22.21	0.4591	0.4213	0.4213
BAD-Gaussians [25]	28.27	0.7125	0.2266	0.2266	23.39	0.5102	0.3034	0.3034	30.24	0.6383	0.4374	0.4374	28.41	0.7041	0.3347	0.3347
Deblurring 3DGS [8]	30.44	0.7708	0.2587	0.2587	26.26	0.6401	0.3964	0.3964	30.87	0.6643	0.5561	0.5561	27.56	0.6559	0.4431	0.4431
BAGS [9]	25.40	0.6142	0.4962	0.4962	23.29	0.5177	0.5450	0.5450	29.06	0.5577	0.6738	0.6738	24.00	0.5402	0.5534	0.5534
CoMoGaussian	31.82	0.8011	0.2170	0.2170	28.53	0.7004	0.2846	0.2846	31.95	0.7166	0.4476	0.4476	29.63	0.7351	0.3474	0.3474
ExbluRF	JARS2				POSTBOX				STONE LANTERN				SUNFLOWERS			
	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	LPIPS↓
Mip-Splatting [2]	22.10	0.5682	0.5817	0.5817	23.19	0.5277	0.5710	0.5710	22.64	0.6004	0.6308	0.6308	25.78	0.6775	0.5020	0.5020
ExbluRF [3]	21.99	0.5736	0.3519	0.3519	23.34	0.5287	0.2978	0.2978	26.18	0.6832	0.4236	0.4236	25.25	0.6765	0.3223	0.3223
BAD-Gaussians [25]	26.27	0.6914	0.3326	0.3326	25.01	0.6264	0.2760	0.2760	25.19	0.6724	0.3794	0.3794	27.82	0.7443	0.2865	0.2865
Deblurring 3DGS [8]	26.76	0.7100	0.3942	0.3942	23.89	0.5563	0.3492	0.3492	23.32	0.6430	0.4687	0.4687	29.75	0.7955	0.3248	0.3248
BAGS [9]	22.20	0.5658	0.5268	0.5268	24.76	0.5891	0.4205	0.4205	22.72	0.5979	0.5556	0.5556	26.14	0.6920	0.4508	0.4508
CoMoGaussian	29.72	0.7699	0.3283	0.3283	29.99	0.7631	0.2479	0.2479	28.66	0.7549	0.3402	0.3402	30.90	0.8062	0.2622	0.2622

5.4.8 Additional Qualitative Results

We provide additional visualization results in Fig. 5.11, which demonstrate that our CoMoGaussian outperforms not only in quantitative metrics but also in qualitative performance.

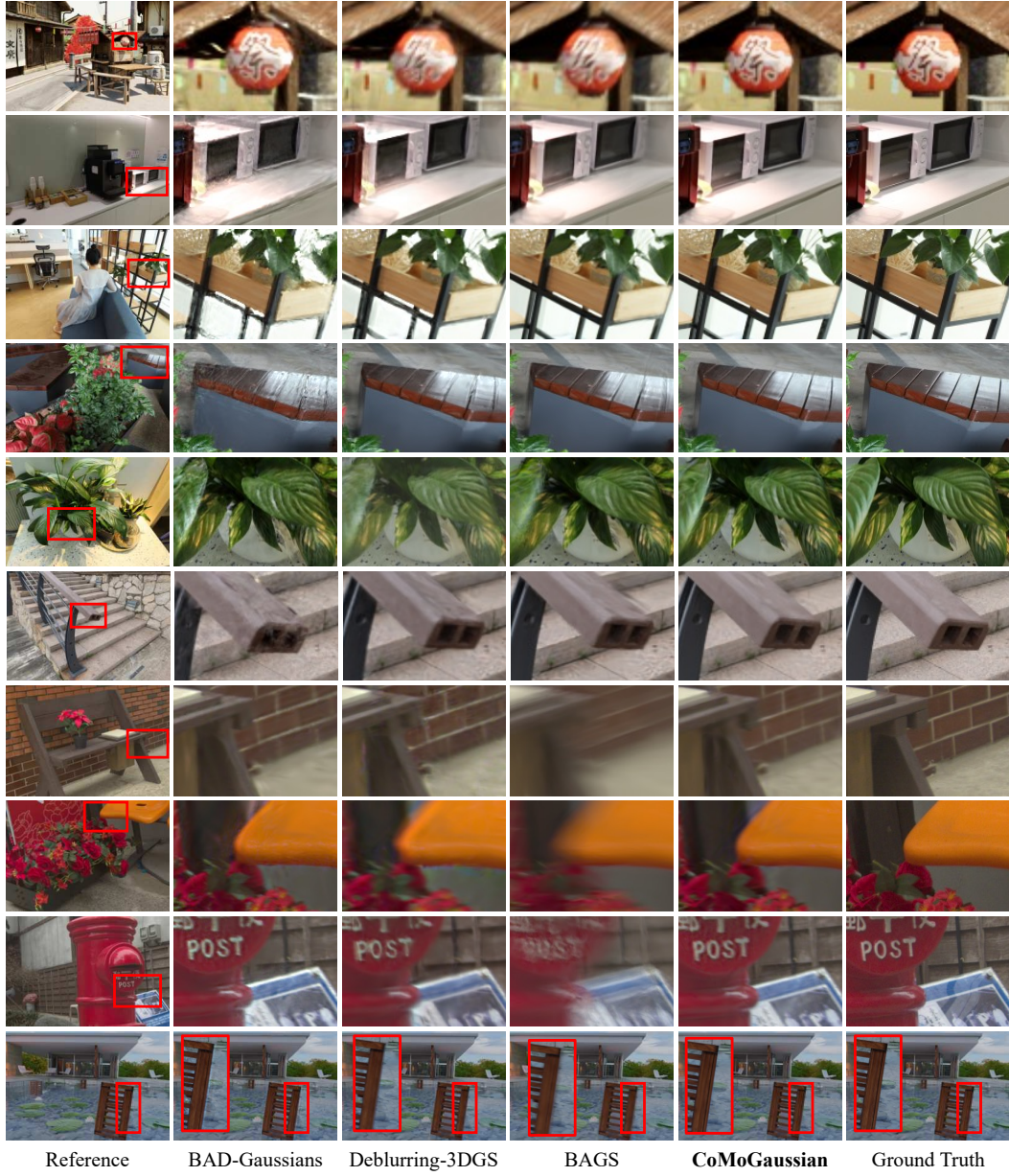


Figure 5.11: Additional Qualitative Comparison on the Synthetic and Real-World Scenes.

Chapter 6

Conclusion

In this thesis, we addressed the problem of learning 3D Gaussian Splatting (3DGS) from degraded images, which is critical for enabling robust neural rendering in real-world scenarios. While existing neural rendering methods typically assume sharp input images, this assumption rarely holds in practice due to various image degradations introduced during the image acquisition process.

To overcome this limitation, we focused on two major types of degradation: defocus blur and motion blur. *For defocus blur*, we proposed a physically grounded framework that explicitly models the Circle of Confusion (CoC) within the 3D Gaussian representation. By leveraging depth-dependent blur modeling and learnable camera parameters, the proposed method enables accurate 3D scene reconstruction using only defocused images while also supporting controllable depth-of-field rendering. *For motion blur*, we introduced a motion-aware Gaussian splatting framework that models the continuous camera trajectory during the exposure time. By incorporating

neural ordinary differential equations (ODEs) to represent time-continuous camera motion and introducing a continuous motion refinement mechanism, the proposed approach accurately captures motion blur formation and reconstructs sharp 3D scenes from motion-blurred inputs.

Through extensive experiments on both synthetic and real-world datasets, we demonstrated that explicitly modeling the physical image formation processes significantly improves reconstruction accuracy and rendering quality. The proposed methods achieve state-of-the-art performance in neural rendering from degraded images while maintaining the efficiency advantages of 3D Gaussian Splatting.

This thesis highlights the importance of integrating physically grounded modeling with efficient explicit scene representations. By bridging the gap between idealized assumptions and real-world imaging conditions, our work advances neural rendering toward more practical and robust applications.

Despite these advancements, several challenges remain. First, real-world images often contain multiple degradations simultaneously, such as defocus blur, motion blur, which are not jointly addressed in this work. Second, extending the proposed frameworks to dynamic scenes and non-rigid motion remains an open problem.

Future research directions include developing unified frameworks that handle multiple degradations simultaneously, extending the methods to dynamic and large-scale scenes, and integrating more comprehensive camera models. We believe that these directions will further enhance the applicability of neural rendering systems in real-world environments.

Bibliography

- [1] L. Ma, X. Li, J. Liao, Q. Zhang, X. Wang, J. Wang, and P. V. Sander, “Deblur-nerf: Neural radiance fields from blurry images,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12861–12870, 2022.
- [2] Z. Yu, A. Chen, B. Huang, T. Sattler, and A. Geiger, “Mip-splatting: Alias-free 3d gaussian splatting,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 19447–19456, 2024.
- [3] D. Lee, J. Oh, J. Rim, S. Cho, and K. M. Lee, “Exblurf: Efficient radiance fields for extreme motion blurred images,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 17639–17648, 2023.
- [4] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” in *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I*, pp. 405–421, 2020.

- [5] B. Mildenhall, P. P. Srinivasan, R. Ortiz-Cayon, N. K. Kalantari, R. Ramamoorthi, R. Ng, and A. Kar, “Local light field fusion: Practical view synthesis with prescriptive sampling guidelines,” *ACM Transactions on Graphics (ToG)*, vol. 38, no. 4, pp. 1–14, 2019.
- [6] Z. Wu, X. Li, J. Peng, H. Lu, Z. Cao, and W. Zhong, “Dof-nerf: Depth-of-field meets neural radiance fields,” in *Proceedings of the 30th ACM International Conference on Multimedia*, pp. 1718–1729, 2022.
- [7] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.
- [8] B. Lee, H. Lee, X. Sun, U. Ali, and E. Park, “Deblurring 3d gaussian splatting,” *arXiv preprint arXiv:2401.00834*, 2024.
- [9] C. Peng, Y. Tang, Y. Zhou, N. Wang, X. Liu, D. Li, and R. Chellappa, “Bags: Blur agnostic gaussian splatting through multi-scale kernel modeling,” *arXiv preprint arXiv:2403.04926*, 2024.
- [10] D. Lee, M. Lee, C. Shin, and S. Lee, “Dp-nerf: Deblurred neural radiance field with physical scene priors,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12386–12396, 2023.
- [11] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, “Neural ordinary differential equations,” *Advances in neural information processing systems*, vol. 31, 2018.

- [12] A. Chen, Z. Xu, A. Geiger, J. Yu, and H. Su, “Tensorf: Tensorial radiance fields,” in *European Conference on Computer Vision*, pp. 333–350, Springer, 2022.
- [13] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, “Plenoxels: Radiance fields without neural networks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5501–5510, 2022.
- [14] A. Yu, R. Li, M. Tancik, H. Li, R. Ng, and A. Kanazawa, “Plenotrees for real-time rendering of neural radiance fields,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5752–5761, 2021.
- [15] M. Niemeyer, J. T. Barron, B. Mildenhall, M. S. Sajjadi, A. Geiger, and N. Radwan, “Regnerf: Regularizing neural radiance fields for view synthesis from sparse inputs,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5480–5490, 2022.
- [16] J. Yang, M. Pavone, and Y. Wang, “Freenerf: Improving few-shot neural rendering with free frequency regularization,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8254–8263, 2023.
- [17] G. Wang, Z. Chen, C. C. Loy, and Z. Liu, “Sparsenerf: Distilling depth ranking for few-shot novel view synthesis,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9065–9076, 2023.

- [18] Z. Wang, S. Wu, W. Xie, M. Chen, and V. A. Prisacariu, “Nerf–: Neural radiance fields without known camera parameters,” *arXiv preprint arXiv:2102.07064*, 2021.
- [19] W. Bian, Z. Wang, K. Li, J.-W. Bian, and V. A. Prisacariu, “Nope-nerf: Optimising neural radiance field with no pose prior,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4160–4169, 2023.
- [20] B. Mildenhall, P. Hedman, R. Martin-Brualla, P. P. Srinivasan, and J. T. Barron, “Nerf in the dark: High dynamic range view synthesis from noisy raw images,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16190–16199, 2022.
- [21] N. Pearl, T. Treibitz, and S. Korman, “Nan: Noise-aware nerfs for burst-denoising,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12672–12681, 2022.
- [22] C. Peng and R. Chellappa, “Pdrf: progressively deblurring radiance field for fast scene reconstruction from blurry images,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 2029–2037, 2023.
- [23] P. Wang, L. Zhao, R. Ma, and P. Liu, “Bad-nerf: Bundle adjusted deblur neural radiance fields,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4170–4179, 2023.
- [24] J. Lee, D. Lee, M. Lee, D. Kim, and S. Lee, “Smurf: Continuous dynamics for motion-deblurring radiance fields,” *arXiv preprint arXiv:2403.07547*, 2024.

- [25] L. Zhao, P. Wang, and P. Liu, “Bad-gaussians: Bundle adjusted deblur gaussian splatting,” *arXiv preprint arXiv:2403.11831*, 2024.
- [26] E. Hecht, *Optics*. Pearson Education India, 2012.
- [27] O. Whyte, J. Sivic, A. Zisserman, and J. Ponce, “Non-uniform deblurring for shaken images,” *International journal of computer vision*, vol. 98, pp. 168–186, 2012.
- [28] A. Chakrabarti, “A neural approach to blind motion deblurring,” in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part III 14*, pp. 221–235, Springer, 2016.
- [29] P. P. Srinivasan, R. Ng, and R. Ramamoorthi, “Light field blind motion deblurring,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3958–3966, 2017.
- [30] J. Lee, D. Kim, D. Lee, S. Cho, and S. Lee, “Crim-gs: Continuous rigid motion-aware gaussian splatting from motion blur images,” *arXiv preprint arXiv:2407.03923*, 2024.
- [31] B. O. Community, *Blender - a 3D modelling and rendering package*. Blender Foundation, Stichting Blender Foundation, Amsterdam, 2018.
- [32] J. L. Schönberger, E. Zheng, J.-M. Frahm, and M. Pollefeys, “Pixelwise view selection for unstructured multi-view stereo,” in *European conference on computer vision*, pp. 501–518, Springer, 2016.

- [33] Q. Shan, J. Jia, and A. Agarwala, “High-quality motion deblurring from a single image,” *Acm transactions on graphics (tog)*, vol. 27, no. 3, pp. 1–10, 2008.
- [34] S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, and M.-H. Yang, “Restormer: Efficient transformer for high-resolution image restoration,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 5728–5739, 2022.
- [35] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [36] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 586–595, 2018.
- [37] E. Lindelöf, “Sur l’application de la méthode des approximations successives aux équations différentielles ordinaires du premier ordre,” *Comptes rendus hebdomadaires des séances de l’Académie des sciences*, vol. 116, no. 3, pp. 454–457, 1894.
- [38] Y. Rubanova, R. T. Chen, and D. K. Duvenaud, “Latent ordinary differential equations for irregularly-sampled time series,” *Advances in neural information processing systems*, vol. 32, 2019.
- [39] S. Park, K. Kim, J. Lee, J. Choo, J. Lee, S. Kim, and E. Choi, “Vid-ode: Continuous-time video generation with neural ordinary differential equation,” in

- Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 2412–2422, 2021.
- [40] E. L. Ince, *Ordinary differential equations*. Courier Corporation, 1956.
- [41] L. Euler, *Institutionum calculi integralis*, vol. 4. impensis Academiae imperialis scientiarum, 1845.
- [42] W. Kutta, *Beitrag zur näherungsweisen Integration totaler Differentialgleichungen*. Teubner, 1901.
- [43] K. Park, U. Sinha, J. T. Barron, S. Bouaziz, D. B. Goldman, S. M. Seitz, and R. Martin-Brualla, “Nerfies: Deformable neural radiance fields,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5865–5874, 2021.
- [44] K. M. Lynch and F. C. Park, *Modern robotics*. Cambridge University Press, 2017.
- [45] O. Rodrigues, *De l’attraction des sphéroïdes, Correspondence sur l’École Impériale Polytechnique*. PhD thesis, PhD thesis, Thesis for the Faculty of Science of the University of Paris, 1816.
- [46] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.

국문 요약

열화된 영상으로부터의 3D Gaussian Splatting 학습

연세대학교 대학원

전기전자공학과

이 정 호

최근 3D Gaussian Splatting은 실시간 뉴럴 렌더링을 위한 효율적인 장면 표현 방식으로 주목받고 있으며, 기존의 암시적 방법에 비해 학습 및 렌더링 속도를 크게 향상시키면서도 고품질의 새로운 시점 영상 생성을 가능하게 한다. 그러나 대부분의 기존 연구는 입력 영상이 선명하다는 가정을 기반으로 하고 있으며, 이는 실제 환경에서는 현실적이지 않다. 실제 영상 획득 과정에서는 제한된 심도에 의해 발생하는 디포커스 블러와 노출 시간 동안 카메라의 움직임으로 인한 모션 블러 등 다양한 열화가 빈번하게 발생한다. 이러한 열화는 정확한 3차원 복원을 어렵게 만들며, 뉴럴 렌더링 기법의 실제 적용 가능성을 제한하는 주요 요인으로 작용한다.

본 논문에서는 열화된 영상으로부터 3D Gaussian Splatting을 학습하는 문제를 다루며, 블러가 존재하는 관측으로부터도 정확한 3차원 복원을 가능하게 하는 물리 기반의 프레임워크를 제안한다. 먼저, 광학적 원리에 기반하여 착란원을 모델링하는 디포커스 인지 Gaussian Splatting 기법을 제안한다. 이를 통해 디포커스 영상만을 이용하여도 정밀한 3차원 장면을 복원할 수 있으며, 학습된 조리개 및 초점면 정보를 활용하여 심도 조절이 가능한 렌더링을 수행할 수 있다. 다음으로, 노출 시간 동안의 카메라 움직임의

로 인해 발생하는 모션 블러를 해결하기 위해, 신경망 미분방정식을 활용하여 연속적인 카메라 궤적을 모델링하는 방법을 제안한다. 또한 연속적인 강체 운동과 모션 보정 변환을 결합하여 모션 블러 형성 과정을 보다 정확하게 반영하고, 모션 블러 영상으로부터 선명한 3차원 장면을 복원할 수 있도록 한다.

제안하는 두 접근법은 실제 영상 획득 과정에서 발생하는 물리적 열화 과정을 명시적으로 모델링하는 것이 비이상적인 환경에서의 강건한 뉴럴 렌더링을 위해 필수적임을 보여준다. 다양한 벤치마크 데이터셋에 대한 실험을 통해, 제안된 방법이 열화된 영상으로부터 고품질의 새로운 시점 영상을 생성하고 정확한 3차원 장면을 복원하는 데 있어 기존 방법 대비 우수한 성능을 달성함을 확인하였다. 본 연구는 물리 기반 영상 형성 모델과 효율적인 명시적 장면 표현을 통합함으로써, 실제 환경에서도 안정적으로 동작하는 3차원 복원 시스템으로의 확장을 가능하게 한다.

핵심되는 말: 3차원 뉴럴 렌더링; *Gaussian Splatting*; 이미지 열화.